

1. Принцип «Соглашение над конфигурацией» в контексте Maven.

Стандартная структура каталогов Maven-проекта.

Принцип «Соглашение над конфигурацией» (Convention over Configuration) в Maven – это концепция, согласно которой процесс создания, сборки и распространения любого проекта (будь то библиотека, веб-приложение или корпоративный модуль) изначально четко определен и стандартизирован.

Суть принципа заключается в том, что разработчику не нужно вручную прописывать в конфигурационных файлах базовые параметры проекта (например, указывать, в какой папке лежит исходный код или куда сохранять скомпилированные файлы). Если структура проекта соответствует принятым в Maven стандартам, система сборки работает автоматически с минимальным объемом файла настроек `pom.xml`.

Стандартная структура каталогов Maven-проекта:

- `project-name/` – корневая папка проекта.
 - `pom.xml` – главный файл конфигурации проекта (Project Object Model), содержащий его координаты, зависимости и плагины.
 - `src/` – директория со всеми исходными материалами проекта.
 - `main/` – исходный код и ресурсы для основной рабочей логики приложения.
 - `java/` – исходный код программы на языке Java.
 - `resources/` – конфигурационные файлы, свойства (`.properties`), настройки логирования и другие статические ресурсы.

- test/ – исходный код и ресурсы, используемые исключительно для тестирования.
 - java/ – файлы тестов (например, JUnit-тесты).
 - resources/ – ресурсы и конфигурации, необходимые только для запуска тестов.
- target/ – каталог, в который помещаются результаты сборки (автоматически генерируется Maven, содержит скомпилированные .class файлы и итоговые исполняемые архивы JAR/WAR).

2. Координаты Maven (<groupId>, <artifactId>, <version>).

Сформулировать правила именования для каждой составляющей.

Координаты Maven (GAV) – это уникальные идентификаторы проекта, которые позволяют однозначно определить любой артефакт (библиотеку, модуль или приложение) в репозитории.

1. <groupId> – Группа / компания

Определяет принадлежность проекта к определенной организации, под разделом или разработчику (например, ru.mtuci или com.example). **Правила именования:**

- Использование только строчных букв (lowercase).
- Использование точек в качестве разделителей элементов структуры (по аналогии с именованием пакетов в Java).
- Полный запрет на пробелы и специальные символы (разрешены только символы из диапазона [a-z0-9_.-]).

2. <artifactId> – Имя артефакта

Задаёт уникальное название конкретного проекта или модуля в рамках своей группы (например, love-Java или my-awesome-app). **Правила именования:**

- Использование только строчных букв (lowercase) – это строгое соглашение.
- Разделение слов выполняется строго с помощью дефисов (использование знаков подчёркивания _ не рекомендуется).
- Разрешены только символы из диапазона [a-z0-9_-].

3. <version> – Версия

Указывает на конкретный этап разработки или релиз проекта (например, 1.0.0 или 1.0.0-SNAPSHOT). **Правила именования:**

- Состоит преимущественно из цифр и разделяющих их точек.
- Буквы допускаются только в качестве специальных маркеров состояния сборки (например: SNAPSHOT, RC, RELEASE).
- Соответствует стандартизированному формату: <major>.<minor>.<patch>[-<qualifier>].

3. Стандартный жизненный цикл сборки Maven. Перечислить основные фазы и описать их назначение.

Стандартный жизненный цикл сборки Maven (Default Lifecycle) представляет собой четкую последовательность шагов, называемых фазами. Главное правило цикла: **запуск любой фазы автоматически запускает все предыдущие фазы по порядку.**

Основные фазы в порядке их выполнения и их назначение:

- **validate** – проверяет корректность файла `pom.xml` и доступность всей необходимой информации для начала сборки.
- **compile** – компилирует исходный код проекта и сохраняет результаты в каталог `target/classes`.
- **test** – запускает тесты (например, JUnit) из директории `src/test/java`.
- **package** – упаковывает скомпилированный код в итоговый распространяемый формат, например, в архив JAR или WAR, создавая его в папке `target`.
- **verify** – проводит проверку качества проекта, в том числе запускает интеграционные тесты.
- **install** – копирует получившийся JAR/WAR-пакет в локальный репозиторий разработчика (`~/.m2/repository`), делая его доступным для других проектов на этом же компьютере.
- **deploy** – отправляет финальный собранный артефакт на удаленный сервер или репозиторий для совместного использования другими разработчиками.

Важное дополнение: Помимо основного жизненного цикла, часто используется независимая команда **clean**, которая полностью удаляет папку `target/` с результатами предыдущих сборок, чтобы начать процесс с чистого листа. Самая популярная комбинация для сборки проекта с нуля – `mvn clean install`.

4. Механизм управления

транзитивными

зависимостями в Maven.

Области видимости

зависимостей: `compile`, `provided`, `runtime`, `test`.

Механизм управления транзитивными зависимостями

Транзитивные зависимости – это механизм, при котором Maven автоматически загружает библиотеки, необходимые для работы тех библиотек, которые вы явно подключили. Если ваш проект зависит от библиотеки А, а библиотека А зависит от библиотеки Б, Maven автоматически добавит библиотеку Б в ваш проект.

Для предотвращения конфликтов версий (когда разные библиотеки требуют разные версии одной и той же транзитивной зависимости) Maven использует три механизма:

1. **Правило кратчайшего пути (Dependency Mediation):** Maven выбирает ту версию, которая находится ближе к корню проекта в дереве зависимостей. Если Проект → А → Б (v1.0) и Проект → В → Г → Б (v2.0), то выберется версия v1.0. Если пути равны, выберется та, которая объявлена в pom.xml раньше.
2. **Исключения (<exclusions>):** Разработчик может вручную запретить загрузку конкретной транзитивной зависимости внутри блока нужной библиотеки.
3. **Управление зависимостями (<dependencyManagement>):** Позволяет жестко зафиксировать версию библиотеки для всего проекта и всех его подмодулей. Если версия указана здесь, Maven проигнорирует транзитивные версии и возьмет именно её.

Области видимости зависимостей (Dependency Scopes)

Область видимости определяет, на каком этапе жизненного цикла (компиляция, тестирование, выполнение) доступна библиотека и попадет ли она в финальный сборщик (JAR/WAR).

- **compile** (основной scope по умолчанию)
 - **Когда доступна:** На всех этапах (компиляция основного кода, тесты, запуск/выполнение).

- **Сборка:** Попадает в финальный JAR/WAR-архив.
- **Транзитивность:** Передается транзитивно в другие проекты, которые используют данный проект.
- **provided (предоставляемый)**
 - **Когда доступна:** Нужна только для компиляции и тестирования. В момент запуска предполагается, что эта библиотека уже есть в среде исполнения (например, в веб-контейнере Tomcat или WildFly). Типичный пример: `servlet-api`.
 - **Сборка:** НЕ попадает в финальный JAR/WAR-архив.
 - **Транзитивность:** Не передается транзитивно.
- **runtime (время выполнения)**
 - **Когда доступна:** НЕ нужна для компиляции (в коде нет прямых вызовов классов этой библиотеки), но обязательна для тестирования и запуска приложения. Типичный пример: JDBC-драйверы баз данных (работа с ними идет через стандартные интерфейсы Java).
 - **Сборка:** Попадает в финальный JAR/WAR-архив.
 - **Транзитивность:** Передается транзитивно.
- **test (тестирование)**
 - **Когда доступна:** Нужна исключительно для написания и запуска тестов (в каталоге `src/test`). Типичные примеры: JUnit, Mockito.
 - **Сборка:** НЕ попадает в финальный JAR/WAR-архив и никак не влияет на работу основного приложения.
 - **Транзитивность:** Не передается транзитивно.

5. Структура

многомодульного проекта

Maven. Различия между

родительским и дочерним POM файлами. Назначение секций <modules> и <dependencyManagement>.

Структура многомодульного проекта Maven

Многомодульный проект состоит из одного главного (родительского) проекта и нескольких вложенных в него подпроектов (модулей). Каждый модуль является самостоятельной единицей (например, один отвечает за логику базы данных, другой – за веб-интерфейс), но их сборка координируется из единого центра.

```
root-project/ (Родительский каталог)
├── pom.xml (Родительский POM)
├── database-module/ (Дочерний модуль 1)
│   ├── pom.xml
│   └── src/
└── web-module/ (Дочерний модуль 2)
    ├── pom.xml
    └── src/
```

Различия между родительским и дочерним POM-файлами

- **Родительский POM (Parent):**
 - Обязательно имеет тип упаковки `<packaging>pom</packaging>`.
 - Не содержит исходного кода (в его папке нет директории `src/main/java`).
 - Служит конфигурационным шаблоном: хранит общие настройки, версии библиотек и плагинов, которые автоматически наследуются всеми модулями.
- **Дочерний POM (Child):**
 - Имеет стандартный тип упаковки (обычно `jar` или `war`).
 - Содержит бизнес-логику и исходный код в каталоге `src`.

- Обязательно содержит блок `<parent>`, где указаны координаты родительского проекта, чтобы связать себя с ним.

Назначение секции `<modules>`

Прописывается только в родительском POM-файле. В ней перечисляются имена папок всех дочерних модулей.

- **Зачем нужна:** Она указывает Maven, какие подпроекты нужно собирать. При запуске команды (например, `mvn clean install`) в корневом каталоге, Maven сканирует эту секцию, анализирует связи между модулями, выстраивает их в правильном порядке (в соответствии с их зависимостями друг от друга) и собирает весь проект за один клик.

Назначение секции `<dependencyManagement>`

Используется для централизованного управления версиями зависимостей (библиотек) во всем многомодульном проекте. Обычно объявляется в родительском POM.

- **Как это работает:** Внутри родительского `<dependencyManagement>` вы жестко фиксируете координаты библиотек и их версии (например, `Spring v6.1.0`). При этом сами библиотеки в проект еще не скачиваются.
- **Зачем нужна:** Когда дочернему модулю требуется эта библиотека, разработчик указывает ее в дочернем POM в обычной секции `<dependencies>`, но не пишет версию. Maven автоматически подтянет правильную версию из родительского файла. Это гарантирует, что все модули гарантированно используют одинаковые версии библиотек, исключая конфликты совместимости.

6. Модель Gradle (Project, Task, Action). Фазы жизненного цикла Gradle.

Модель Gradle (Project, Task, Action)

Gradle представляет процесс сборки в виде направленного ациклического графа (DAG), узлами которого являются задачи (Tasks), а связями – зависимости между ними.

- **Project (Проект / интерфейс Project):** Главный элемент сборки. Проектом в Gradle называется любая часть программного обеспечения, которую можно собрать (само приложение, библиотека, отдельный модуль). Каждому конфигурационному файлу `build.gradle` / `build.gradle.kts` строго соответствует один объект `Project` в коде Gradle.
- **Task (Задача / интерфейс Task):** Базовая атомарная единица работы в рамках проекта (например, компиляция кода `compileJava`, запуск тестов `test`, или упаковка в архив `jar`). Задачи могут зависеть друг от друга, определяя порядок сборки. Каждая задача физически состоит из последовательности действий (Action).
- **Action (Действие / интерфейс Action):** Минимальный неделимый шаг внутри конкретной задачи, представляющий собой исполняемый программный код (метод `execute()`). Задача может содержать несколько действий (например, выполняемые через блоки `doFirst` и `doLast`).

Фазы жизненного цикла Gradle

При любом запуске Gradle сборка всегда последовательно проходит через 3 обязательные фазы:

1. Инициализация (Initialization):

- Gradle ищет файл настроек `settings.gradle` / `settings.gradle.kts` в корневом каталоге.
- Создается объект `Settings`, скрипт файла настроек выполняется, чтобы определить структуру сборки (какие именно подпроекты и модули в неё входят).
- Для каждого обнаруженного проекта создается пустой экземпляр объекта `Project`.

2. Конфигурация (Configuration):

- Gradle последовательно выполняет скрипты `build.gradle` всех проектов, участвующих в сборке.
- На основе кода этих скриптов настраиваются свойства объектов `Project` и создаются задачи (`Tasks`).
- В конце фазы Gradle строит полный граф зависимостей задач (определяет, какие задачи в каком порядке должны быть выполнены для достижения цели).
- *Важно:* Код, написанный в теле задачи вне блоков `doFirst/doLast`, выполняется именно на этой фазе.

3. Исполнение / Выполнение (Execution):

- Gradle отбирает из построенного графа только те задачи, которые были запрошены пользователем (и их транзитивные зависимости).
- Gradle последовательно запускает выбранные задачи и выполняет содержащиеся в них действия (`Actions`).
- Благодаря механизму инкрементальной сборки, если входные и выходные файлы задачи не изменились с прошлого раза, её выполнение пропускается (UP-TO-DATE).

7. Конфигурации зависимостей Gradle.

Конфигурация зависимостей в Gradle – это именованная группа зависимостей, которая определяет область их видимости и то, на каком этапе (компиляция основного кода,

запуск тестов или выполнение программы) эти библиотеки будут доступны.

В Gradle конфигурации заменяют механизм scope из Maven и могут наследоваться друг от друга, формируя пути к классам (classpath).

Основные конфигурации для исходного кода (src/main)

- **implementation** (основная и самая частая)
 - **Назначение:** Библиотека нужна и для компиляции, и для работы приложения.
 - **Особенность:** Скрывает внутренние зависимости модуля. Если ваш модуль А использует библиотеку через **implementation**, то модуль Б, который зависит от А, эту библиотеку автоматически при компиляции не увидит (инкапсуляция сборки, ускоряет компиляцию).
 - **Сборка:** Попадает в финальный архив (JAR/WAR).
- **api** (доступна при подключении плагина `java-library`)
 - **Назначение:** Библиотека нужна для компиляции и работы, но в отличие от **implementation** она является «публичной».
 - **Особенность:** Транзитивно передается выше. Если модуль А подключил библиотеку через **api**, то модуль Б, зависящий от А, получит к ней прямой доступ при своей компиляции.
- **compileOnly**
 - **Назначение:** Библиотека нужна исключительно на этапе компиляции кода (чтобы проект успешно собрался), но не требуется при запуске.
 - **Пример:** Аннотации Lombok (`org.projectlombok:lombok`) или API сервлетов, которое уже встроено в сервер приложений.
 - **Сборка:** НЕ попадает в финальный архив.

- runtimeOnly
 - **Назначение:** Библиотека НЕ нужна для компиляции (в коде нет прямого импорта классов этой библиотеки), но обязательна для работы запущенной программы.
 - **Пример:** Драйверы баз данных (например, org.postgresql:postgresql). Код пишется под стандартный интерфейс JDBC, а конкретная реализация подкладывается на этапе выполнения.
 - **Сборка:** Попадает в финальный архив.
- annotationProcessor
 - **Назначение:** Используется для подключения плагинов и библиотек, которые генерируют код на этапе компиляции на основе аннотаций (например, тот же Lombok или MapStruct).

Конфигурации для тестирования (src/test)

Полностью дублируют логику основных конфигураций, но работают только в рамках выполнения тестов и никогда не попадают в готовый продукт:

- testImplementation – библиотеки для компиляции и запуска тестов (например, JUnit, Mockito).
- testCompileOnly – нужны только для компиляции тестов.
- testRuntimeOnly – нужны только в момент выполнения тестов.

8. Принципу Inversion of Control.

Проблемы традиционного подхода к созданию объектов, решаемые применением IoC.

Принцип Инверсии управления (Inversion of Control, IoC) – это архитектурный принцип, при котором управление жизненным циклом объектов и потоком выполнения программы

передается от самого приложения внешнему каркасу (фреймворку или IoC-контейнеру). При традиционном подходе код разработчика управляет сторонними библиотеками, а при IoC – фреймворк сам управляет кодом приложения (так называемый «Голливудский принцип»: *Не звоните нам, мы сами вам позвоним*).

Проблемы традиционного подхода к созданию объектов (через оператор `new`)

1. Жесткая связанность (Tight Coupling):

Если класс `Car` сам внутри себя создает зависимость через `Engine eng = new V8Engine();`, он намертво привязан к конкретной реализации. Если завтра понадобится заменить двигатель на `ElectricEngine`, придется переписывать и заново компилировать код класса `Car`.

2. Сложность тестирования (Difficult Testing):

Невозможно протестировать класс изолированно (с помощью заглушек или `mock`-объектов). Если зашитый внутри `V8Engine` делает реальные запросы в базу данных или сеть, то юнит-тест для `Car` будет зависеть от внешних факторов, станет медленным и нестабильным.

3. Нарушение принципа единственной обязанности (Single Responsibility Principle):

Класс должен отвечать исключительно за свою бизнес-логику. При ручном создании объектов класс вынужден брать на себя лишнюю ответственность: знать, как правильно сконструировать, настроить и вовремя уничтожить свои зависимости.

4. Хаос в управлении жизненным циклом:

Сложно контролировать количество созданных объектов в масштабах всего

приложения. Например, тяжело гарантировать, что какой-то тяжелый сервис существует строго в одном экземпляре (паттерн Singleton), если любой разработчик в команде может вызвать new.

Как IoC решает эти проблемы

При использовании IoC объект полностью освобождается от обязанности создавать свои зависимости. Он лишь декларирует потребность: *«Мне для работы нужен любой объект, реализующий интерфейс Engine»*.

Специальный компонент фреймворка (IoC-контейнер) берет на себя всю рутину:

- Сам находит или создает нужный объект-зависимость.
- Сам настраивает его.
- Сам передает (внедряет) его в требуемый класс через конструктор или сеттер.

Результат: Связанность компонентов становится минимальной (Loose Coupling), код легко расширяется, а для тестирования достаточно передать в конструктор тестируемого класса простой mock-объект.

9. Внедрение зависимостей как способ реализации IoC. Типы внедрения зависимостей.

Внедрение зависимостей (Dependency Injection, DI) – это конкретный паттерн проектирования, с помощью которого реализуется архитектурный принцип IoC. Если IoC – это общая концепция (передача контроля фреймворку), то DI – это практический механизм ее выполнения. Вместо того чтобы объект сам создавал нужные ему ресурсы, внешняя система (IoC-контейнер) создает эти ресурсы и «внедряет» (передает) их в объект.

Основные типы внедрения зависимостей

1. Внедрение через конструктор (*Constructor Injection*)

Зависимости передаются в класс в качестве обязательных параметров его конструктора в момент создания объекта.

- **Преимущества:**

- Гарантирует, что объект нельзя создать в «полуготовом» состоянии – он сразу инициализирован всеми зависимостями.
- Позволяет объявлять поля зависимостей как `final` (неизменяемые), что обеспечивает потокобезопасность.
- Код легко тестировать в юнит-тестах: можно вручную передать `mock`-объекты через `new MyService(mockDependency)`.

- **Применение:** Самый рекомендуемый и стандартный способ в современной разработке.

2. Внедрение через метод изменения / сеттер (*Setter Injection*)

Сначала контейнер вызывает конструктор (обычно без параметров) для создания самого объекта, а затем последовательно вызывает специальные методы (сеттеры) для передачи в него зависимостей.

- **Преимущества:**

- Позволяет делать зависимости необязательными.
- Дает возможность изменять или перенастраивать зависимости «на лету» во время работы программы.

- **Недостатки:**

- Объект временно находится в нестабильном (не полностью собранном) состоянии между вызовом конструктора и вызовом сеттера.
- Поля зависимостей невозможно сделать `final`.

3. Внедрение напрямую в поле (Field Injection)

Зависимости внедряются непосредственно в приватные поля класса в обход конструкторов и сеттеров с помощью механизмов рефлексии (Java Reflection API). В Spring для этого над полем указывается аннотация `@Autowired`.

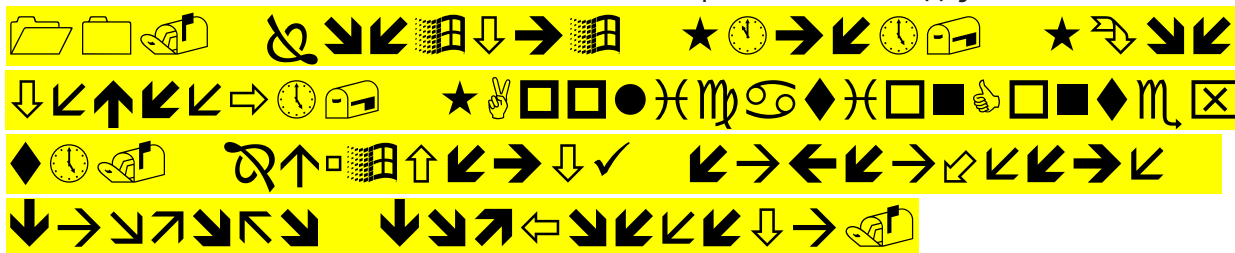
- **Преимущества:**

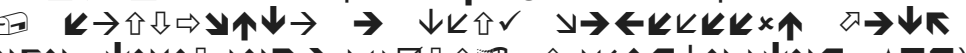
- Код выглядит очень лаконично и компактно, не нужно писать конструкторы или сеттеры.

- Недостатки:

- Объект становится жестко привязан к IoC-контейнеру. Без запуска фреймворка этот класс невозможно использовать.
- Невозможно сделать поля `final`.
- Существенно усложняется написание изолированных юнит-тестов (приходится задействовать рефлексии или Mockito, чтобы внедрить заглушку в приватное поле).

- **Применение:** В современном Java-сообществе считается антипаттерном, использовать его не рекомендуется.

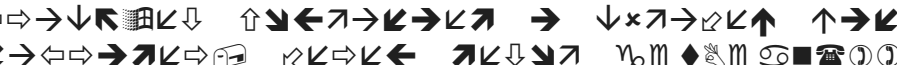


- 



- 
- 
- 
- 



- 
- 
- 



[illegible]



A set of navigation icons including arrows, a magnifying glass, and other symbols.


































































































































































[illegible]











































































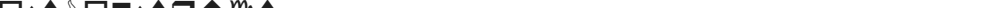






⌘→←↶→↷↶→↷ →↶↷↷↓→↻→↑ ↺↻□♦♣□■♦□◆♠♦ → ↺↻□♠♣

♠♦◆□□☒

- 
 - 

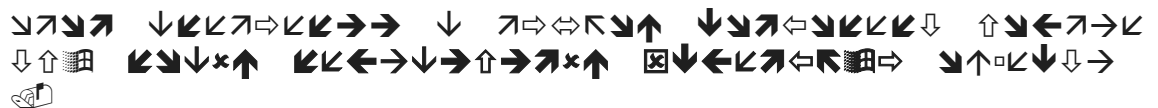
- ↓→↓ ♠□✕■♪. ↑↘←→↖ ↘↑↗↓↓ → ⇌↑↖↘↗ ↓
 ↗↖→↖ ↓ ↗↖ ↘↑↗ ←→↓→↑→↗↑↓→👉
- 📞→↗↖ ↗↖↗📞 ⌚ ↘↑↗↗↖ ↓↗↑↓⇌↓↘↗↖ ☺
 📞📞📞 →↑↖↗↖↖↗↓→↓↖ ↖↗📞 ↑ →↗↗↓→↗→↗↑
 📞📞♦♦□♦✕□♣ ♠ ↗↖↖↖↖ ↓→↓ ↓→↓ ↓ ↗↗↗↗↓ ↓
 ✕↖↘↓→ ↓↗↑↓⇌↓↘↗↖→ ↗↖ ↗↖ ⇌→↓↗✕ ■♦●●
 👉 ⌚ ↗↗↗↗ 📞📞□♦📞□♦♦□♦♣ ↓↑↗ ←→↓→↑→↗
 ↘↑↓→ ↖→⇌→↗↓→⇌↘↓→↗↗ ↖↗↖↖↖↖↗✕👉 📞↗↗↑
 ✓ ↓✕↖↗↗↖↖↖↖↑↓⇌↖↗↖↖↖↖↖ ↖↗↖↖↖↖ ↖↗↖↖↖↖
 →↗→↖↖↖↖↖↖↖ ↖↖↖↖↖↖↖↖↖ ↗→↗↗✕↖ ↓ 📞📞📞 ↖↖↖
 ↓↗↖↖↖ ↘↖↖↖↖↖↖↖↑↓→ ↗→↑↓⇌↖↖↖ →↖→ ←→↖↖
 ↑↓ ↖↗↗↖↖↖↖↖ ↖↖↖↖↖↖↖↖↖↖
 - 📞📞□♣📞♣♦♦□□📞
 - 📞→↖↗↖↖↖↖↖ 📞↗↖↖↖↖↖↖ ↗↖↖↖↖ ↘↖↖↖↖↖↖
 →↓↖↖↖↖↖↖↖↖↖↖ ↘✕↖↖↖↖↖↖↖↖ ↗↖↖↖↖↖↖↖↖↖↖
 ↗↖ ↖↖↖↖↖ ↖↖↖↖ ↘→↓ ↘↖↖↖↖↖↖↖↖ ↖↖→↖↖↖↖↖
 ↓ ↑→↗ 📞↖↖→ ↖↖→↖↖↖↖↖ ↘↑↓→↖↖↖↖↖ ↖↖↖↖↖↖
 →👉↖↖→↖↖↖↖↖↖📞📞📞
 - 📞→↗↖↖ ↗↖↖↖↖📞 📞↑↖↗↖↖↖↖↖↖ ↖↖↖ ↖↖↖↖↖↖
 ↑↖↖↖↖ → ↘↖↖↖↖↖↖↖↖↖ ↘↑↖↖↖↖↖↖↖↖↖↖ ↖↖↖↖
 ↖↖↖↖ 📞♣♣♣📞 ♦♦📞📞 ⌚ 📞↖↖↖ ↗↖↖↖↖ ←→↓↖
 ✕↓→📞↖ ↘↖↖↖↖↖↖↖↖ ↖↖↖↖↖↖ ↖↖↖↖↖↖↖↖↖↖ ↘↑↓
 →↗→↖↖↖↖↖↖↖ →↖↖↖↖↖↖↖ ↖↖↖↖↖↖↖ ↖↖↖↖↖↖↖
 ↓ ↖→↑↖✕ →↖→ ↖↖↖↖↖↖↖↖↖↖ ↖↖↖↖↖↖↖↖↖↖↖↖ ↖
 →↗↖↖↖ →↖ ↖↖↖↖↖↖↖↖↖↖ ↖↖↖↖↖↖↖ ↖ ↖→↖↖↖
 ↖→↗↖↖↖↖📞

📁📄👉 📞↘↖↖↖↖↖ ↖→↗↖↖↖↖ ↖↖↖↖↖↖ ↖↖↖↖↖↖
 □□📞 □□□♦□♦📞□♣📞 □♣□♦♣♦📞♦
 ♣♦♦✕□■👉 📞→↖↖↖↖↖↖↖ ↗↖↖↖↖ ↖↖↖
 ♣♣♣♦□♦📞 → □□□♦□♦📞📞 📞↖↖↖↖
 ↗↖↖↖↖↖↖ ↖↖↖↖↖↖↖↖↖↖ □□□♦□♦
 📞□♣📞📞↖↖↖↖↖↖📞

📞↖↖↖↖↖↖ ↘↖↖↖↖↖↖↖ 📞♣♣□♣♣♦📞 ↖→↗↖↖ ↘ ♠□
 ✕■♪

📞↖↖↖↖↖↖ ↘↖↖↖↖↖↖↖ ↖↖↖↖↖↖↖↖↖↖ ↘→↓ ↗↖↖↖ ↘→↓
 ↗↖ ↘↑↗↖↖ ↖→↗ → ↓ ↘→↓↖↖ ↖→↓↖↖↖↖↖↖ ♠□✕■♪ ↘
 ↗↖↖↖↖↖↖ ↖↖↖↖↖ ↖↖↖↖↖↖↖↖↖↖ ↗↖ ↗↖↖↖↖ 📞↖↖↖↖↖↖
 ✕👉

- ✕■♪♣♣♦□ 📞↖↖↖↖↖↖ ↖↖ ↖↖↖↖↖↖↖↖↖📞📞 📞 ↓ ⇌↖
 ↖↖↖↖ ↖↖↖↖ 📞📞📞↖↖↖↖↖↖↖↖↖ ↖↖↖↖↖ ↖↖↖↖↖↖
 ↑↖↖ ↖↖↖↖ ↖↖→↖ 📞↖↖↖↖↖↖↖↖↖ ↖→↗↖↖↖↖
 ⌚↑↖ ↖↖↖↖↖ ↖→↗✕📞 ←→↖↖→↖↖↖↖↖↖↖↖ 📞↖↖ ←→↓↖
 ↖→↖↖↖↖↖↖↖ ↖↖↖↖↖↖↖↖ ↖↖↖↖↖↖ ↗↖ ↖↖↖↖
 ↗↖ ↘↑↗↖↖📞
- □□□♦□♦📞📞 📞 ↖↖→ ↘↖↖↖↖ ↖↖↖↖↖↖↖ ↖→↗↖ →↖
 ↘↖↖↖↖↖↖↖ 📞↖↖↖↖↖ ♣♣♦♣♣📞📞📞📞📞 →↖→ ↖↖→ ↘→



- request (только для веб-приложений) – один экземпляр бина создается на каждый отдельный HTTP-запрос клиента. Бин уничтожается сразу после завершения обработки этого запроса.
- session (только для веб-приложений) – один экземпляр бина создается на всю HTTP-сессию конкретного пользователя. Он живет в памяти до тех пор, пока сессия активна или не истечет ее таймаут.

Главные различия между singleton и prototype

Критерий	singleton	prototype
Количество объектов	Один на все приложение.	Множество (новый объект на каждый запрос).
Момент создания	При старте приложения (по умолчанию).	Только в момент явного обращения/запроса.
Хранение данных (State)	Обычно stateless (не хранит изменяемые данные, так как объект общий для всех потоков).	Может быть stateful (хранит уникальное состояние для конкретной задачи/пользователя).

Особенности уничтожения prototype-бинов

Это важнейший нюанс для экзамена: **Spring HE** управляет полным жизненным циклом бинов со скоупом **prototype**.

Контейнер берет на себя создание объекта, внедрение в него зависимостей и вызов методов инициализации (включая `@PostConstruct`), после чего просто отдает объект клиенту и «забывает» о нем.

Следствие:

- Методы, помеченные аннотацией `@PreDestroy`, а также интерфейс `DisposableBean` для prototype-бинов автоматически контейнером никогда не вызываются.
- Очистку памяти (работу `Garbage Collector`) и ручное освобождение занятых бином ресурсов (заккрытие открытых файлов, сетевых соединений или потоков) должен полностью контролировать сам разработчик в коде приложения.

13. Алгоритм разрешения зависимостей при использовании аннотации `@Autowired`. Способы разрешения неоднозначности с помощью `@Primary` и `@Qualifier`.

Алгоритм поиска и разрешения зависимости при `@Autowired`

Когда Spring встречает аннотацию `@Autowired`, он выполняет следующие шаги:

1. **Поиск по типу (By Type):** Контейнер ищет в контексте все бины, которые соответствуют или наследуют требуемый класс или интерфейс.
2. **Проверка количества найденных бинов:**
 - Если найден ровно один бин – он успешно внедряется.

- Если бинов не найдено – выбрасывается исключение `NoSuchBeanDefinitionException` (сборка падает).
 - Если найдено несколько подходящих бинов – возникает ситуация неоднозначности, и Spring запускает алгоритм уточнения.
3. **Разрешение неоднозначности (шаги выполняются строго по очереди до первого совпадения):**
- **Шаг 3.1. Проверка `@Qualifier`:** Если в месте внедрения указана аннотация `@Qualifier("имя")`, Spring ищет бин с этим конкретным именем.
 - **Шаг 3.2. Проверка `@Primary`:** Если квалификатора нет, Spring проверяет сами бины-кандидаты. Если один из них помечен аннотацией `@Primary`, выбирается он.
 - **Шаг 3.3. Проверка по имени поля/аргумента (By Name):** Если аннотаций нет, Spring сравнивает имя переменной (или аргумента конструктора/метода) с именами (ID) доступных бинов в контексте. При совпадении внедряется этот бин.
4. **Финальная ошибка:** Если ни один из шагов разрешения неоднозначности не помог сделать однозначный выбор, выбрасывается исключение `NoUniqueBeanDefinitionException`.

Способы разрешения неоднозначности

- **Аннотация `@Primary`**
 - **Где ставится:** Над классом бина (компонентом) или над методом `@Bean` в конфигурации.
 - **Как работает:** Помечает бин как «главный» или «выбираемый по умолчанию». Если для внедрения подходят несколько компонентов, Spring без лишних вопросов выберет тот, у которого стоит `@Primary`.

- *Ограничение:* Для одного типа данных может быть объявлен только один @Primary-бин.
- **Аннотация @Qualifier**
 - *Где ставится:* В месте внедрения зависимости (над приватным полем, перед аргументом конструктора или метода) совместно с @Autowired.
 - *Как работает:* Осуществляет точечное и жесткое связывание. В параметрах аннотации указывается строковое имя (ID) конкретного бина, который необходимо внедрить (например, @Qualifier("digitalCamera")).
 - *Приоритет:* Имеет наивысший приоритет и перекрывает действие аннотации @Primary.

14. Способы

конфигурирования Spring-контейнера. Области применения каждого способа.

В Spring существует три основных способа конфигурирования IoC-контейнера. В современных проектах они часто комбинируются.

1. Конфигурация на основе XML (XML-based Configuration)

- **Суть:** Все бины, их свойства и связи между ними жестко описываются в текстовых XML-файлах (например, applicationContext.xml) с помощью тегов <bean>, <property>, <constructor-arg>.
- **Область применения:** * Поддержка старых проектов (Legacy-код), написанных до появления Spring 3.
 - Ситуации, когда необходимо изменять структуру приложения и связи между объектами «на лету» без

перекомпиляции Java-кода (достаточно поправить XML-файл на сервере).

2. Конфигурация на основе аннотаций (Annotation-based Configuration)

- **Суть:** Классы приложения помечаются специальными аннотациями-стереотипами (`@Component`, `@Service`, `@Repository`, `@Controller`), а связи между ними устанавливаются автоматически с помощью `@Autowired`. Для работы требуется запустить механизм сканирования пакетов (`@ComponentScan`).
- **Область применения:**
 - Написание собственной бизнес-логики приложения.
 - Это стандарт де-факто для современной разработки. Позволяет видеть настройки компонента прямо в его коде, что сокращает объем файлов конфигурации и ускоряет написание кода.

3. Конфигурация на основе Java-кода (Java-based Configuration / JavaConfig)

- **Суть:** Создаются специальные управляющие Java-классы, помеченные аннотацией `@Configuration`. Внутри них пишутся обычные Java-методы, которые создают, настраивают объекты вручную и помечаются аннотацией `@Bean`. Контейнер перехватывает вызовы этих методов и регистрирует возвращаемые объекты как бины.
- **Область применения:**
 - Интеграция сторонних библиотек. Если класс находится внутри подключенного JAR-архива, вы не можете поставить над ним аннотацию `@Component`. Единственный способ сделать его бином – создать его через метод `@Bean` в конфигурационном классе.
 - Реализация сложной или динамической логики создания

- объектов (например, когда создание бина зависит от условий: операционной системы, значений в файле настроек или переменных окружения).
- Централизованная настройка глобальных модулей (безопасность, базы данных, интеграционные потоки).

15. Назначение JDBC.

Перечислить недостатки прямого использования JDBC в прикладном коде.

Назначение JDBC

JDBC (Java Database Connectivity) – это стандартный программный интерфейс (API) для взаимодействия Java-приложений с реляционными базами данных. Он предоставляет унифицированный набор интерфейсов, благодаря которому приложение может отправлять SQL-запросы и получать данные из любой СУБД (PostgreSQL, MySQL, Oracle), не меняя свой код – достаточно лишь подложить нужный драйвер этой СУБД.

Недостатки прямого использования JDBC в прикладном коде

1. **Огромный объем шаблонного кода (Boilerplate code):** Для выполнения даже самого простого SQL-запроса разработчику нужно вручную открыть соединение, создать объект запроса, выполнить его, обработать результат и в строгом порядке закрыть все ресурсы. Логика инфраструктуры занимает 80% кода, а сама бизнес-логика – лишь 20%.
2. **Сложная и громоздкая обработка ошибок:** Все методы JDBC выбрасывают проверяемое исключение `SQLException`. Работа с ними заставляет загромождать код вложенными блоками `try-catch-finally` (или

- сложными конструкциями try-with-resources), что сильно ухудшает читаемость кода.
3. **Ручное связывание данных (Ручной маппинг):** JDBC возвращает результаты в виде таблицы ResultSet. Перенос данных из нее в обычные Java-объекты (POJO) выполняется полностью вручную: приходится писать много строк вида `user.setName(rs.getString("name"))` для каждого поля. Если структура таблицы меняется, приходится переписывать этот код.
 4. **Отсутствие проверки SQL на этапе компиляции:** SQL-запросы пишутся внутри Java-кода в виде обычных текстовых строк. Компилятор Java не знает синтаксиса SQL, поэтому любые опечатки в названиях таблиц или колонок обнаружатся только во время работы программы (в Runtime) в виде падения приложения.
 5. **Высокий риск утечки ресурсов (Resource Leaks):** Если разработчик по ошибке забудет закрыть Connection, Statement или ResultSet, эти объекты останутся висеть в памяти и удерживать соединения с БД. При высокой нагрузке пул соединений СУБД быстро переполнится, и база данных перестанет отвечать.

16. Технология Object-Relational Mapping. Описать, как ORM преобразует Java-классы и объекты в таблицы и строки реляционной БД.

ORM (Object-Relational Mapping – объектно-реляционное отображение) – это технология, которая связывает объектно-ориентированную модель данных приложения (классы и объекты в Java) с реляционной моделью данных (таблицы и строки в СУБД). Она устраняет «несоответствие парадигм», позволяя разработчику работать с базой данных в терминах объектно-ориентированного

программирования (ООП), не написав при этом ни одной строки SQL-кода вручную.

Как именно ORM преобразует Java-компоненты в элементы БД:

1. Класс → Таблица (Class to Table)

Java-класс, предназначенный для хранения в БД, помечается аннотацией `@Entity`. ORM-фреймворк (например, Hibernate) воспринимает этот класс как схему для создания таблицы. По умолчанию имя таблицы совпадает с именем класса, но его можно изменить вручную с помощью аннотации `@Table(name = "users")`.

2. Поле класса → Колонка таблицы (Field to Column)

Приватные поля класса автоматически преобразуются в столбцы этой таблицы. ORM транслирует типы данных Java в соответствующие типы SQL:

- String превращается в VARCHAR.
- Integer / Long превращаются в INT / BIGINT.
- LocalDate / LocalDateTime превращаются в DATE / TIMESTAMP.

Настройки конкретной колонки (ограничение по длине, запрет на null) задаются аннотацией `@Column`.

3. Объект (экземпляр класса) → Строка таблицы (Object Instance to Row)

Конкретный созданный в памяти Java-объект (например, `User user = new User(1L, "Sanya");`) представляет собой одну физическую строку (запись) внутри таблицы базы данных. Когда вы вызываете метод сохранения объекта, ORM автоматически генерирует SQL-запрос INSERT и переносит значения полей объекта в ячейки новой строки БД. При извлечении данных из БД (запрос SELECT) ORM выполняет обратный

процесс – считывает строку таблицы и сама конструирует из неё Java-объект.

4. Поле идентификатора → Первичный ключ (ID to Primary Key)

В реляционных базах данных каждая строка должна иметь уникальный идентификатор. В Java-классе одно из полей обязательно помечается аннотацией `@Id`. В таблице БД это поле становится первичным ключом (PRIMARY KEY). С помощью аннотации `@GeneratedValue` настраивается автоматическая генерация этого ключа на стороне СУБД (например, через механизмы AUTO_INCREMENT или SEQUENCE).

5. Связи между объектами → Внешние ключи (Relationships to Foreign Keys)

В Java связи между объектами выражаются через прямые ссылки (например, у класса `Book` есть поле типа `Author`). В реляционной БД связи реализуются через внешние ключи (FOREIGN KEY). ORM преобразует объектные связи в реляционные с помощью специальных аннотаций:

- `@OneToOne` (Один к одному).
- `@ManyToOne` (Многие к одному) – создает колонку с внешним ключом в текущей таблице.
- `@OneToMany` (Один ко многим).
- `@ManyToMany` (Многие ко многим) – автоматически создает под капотом скрытую промежуточную таблицу для связи двух сущностей.

17. Различия между спецификацией JPA и её реализацией Hibernate. Объяснить их взаимосвязь.

Взаимосвязь JPA и Hibernate

Наилучший способ понять их взаимосвязь – это аналогия из стандартной Java: **JPA** – это интерфейс, а **Hibernate** – это конкретный класс, который его реализует.

- **JPA (Jakarta Persistence API / ранее Java Persistence API)** – это спецификация (стандарт, свод правил и требований). Она представляет собой набор интерфейсов (например, `EntityManager`) и аннотаций (например, `@Entity`, `@Id`, `@Table`), которые только описывают, как должно работать объектно-реляционное отображение, но сами по себе не содержат никакого исполняемого кода. JPA нельзя скачать и запустить.
- **Hibernate** – это реализация (конкретная библиотека/фреймворк). Разработчики Hibernate написали реальный Java-код, который берет аннотации JPA, обрабатывает их, генерирует SQL-запросы, открывает соединения с базой данных и выполняет всю реальную работу под капотом.

Главные различия между JPA и Hibernate

К р и т е р и й	JPA	Hibernate
Что это такое	А б с т р а к т н а я с п е ц и ф и к а ц и я (с т а н д а р т).	К о н к р е т н а я п р о г р а м м н а я б и б л и о т е к а (ф р е й м в о р к).
Из чего состоит	Н а б о р и н т е р ф е й с о в и а н н о т а ц и й (<code>jakarta.persistence.*</code>).	Р е а л ь н ы й и с п о л н я е м ы й к о д, JAR-ф а й л ы (<code>org.hibernate.*</code>).
К т о р а з р а б а т ы в а е т	К о м и т е т (ранее Sun/Oracle, с е й ч а с Eclipse Foundation).	К о м п а н и я Red Hat и open-source с о о б щ е с т в о.

К р и т е р и й	JPA	Hibernate
Г л а в н ы й у п р а в л я ю щ и й к л а с с	EntityManager	Session (к о т о р ы й в н у т р и с е б я р е а л и з у е т EntityManager).
П е р е н о с и м о с т ь к о д а	Е с л и к о д н а п и с а н с т р о г о н а JPA, в ы м о ж е т е л е г к о з а м е н и т ь Hibernate н а д р у г у ю р е а л и з а ц и ю (н а п р и м е р, EclipseLink) б е з и з м е н е н и я к о д а.	Е с л и в ы и с п о л ь з у е т е с п е ц и ф и ч е с к и е ф у н к ц и и Hibernate, п р о е к т с т а н е т ж е с т к о п р и в я з а н к э т о й б и б л и о т е к е.

Почему в реальных проектах используют JPA через Hibernate?

Если писать код, ориентируясь на интерфейсы JPA, приложение становится гибким и стандартизированным. При этом под капотом всю тяжелую работу выполняет Hibernate.

Кроме того, Hibernate появился задолго до JPA и имеет свои **собственные расширенные возможности**, которых (пока) нет в стандарте JPA (например, аннотации `@UpdateTimestamp` для автоматической записи времени изменения строки, более продвинутый механизм кэширования второго уровня или поддержка специфических типов данных конкретных СУБД).

18. Сущность (Entity). Правила создания JPA-сущности

Сущность (Entity). Понятие

Сущность (Entity) – это обычный Java-класс (POJO), который сопоставляется (маппится) с определенной таблицей в базе данных.

- Сам класс задает структуру и схему будущей таблицы.
- Каждый конкретный объект (экземпляр) этого класса представляет собой одну физическую строку (запись) внутри этой таблицы.

Правила создания JPA-сущности

Согласно спецификации JPA (Java Persistence API), к классу сущности предъявляется ряд обязательных и строгих требований:

1. **Наличие аннотации @Entity:** Класс должен быть явно помечен аннотацией @Entity (из пакета jakarta.persistence или старого javax.persistence) либо быть описан в XML-файле конфигурации JPA.
2. **Конструктор без аргументов:** Класс должен обязательно содержать конструктор без параметров (дефолтный конструктор) со спецификатором доступа public или protected. Он необходим ORM-фреймворку (например, Hibernate), чтобы создавать объект сущности через рефлексию при чтении данных из базы. Другие конструкторы с параметрами при этом иметь разрешается.
3. **Класс верхнего уровня:** Entity-класс должен быть классом верхнего уровня (Top-level class), то есть он не может быть внутренним (inner) или вложенным классом.
4. **Не может быть интерфейсом или enum:** Сущность может быть только полноценным классом. Перечисления (enum) или интерфейсы в качестве сущностей использовать нельзя.
5. **Не может быть финальным (final):** Класс не должен содержать модификатор final. Это связано с тем, что Hibernate для реализации механизмов «ленивой загрузки» (Lazy Loading) динамически создает классы-прокси,

- которые наследуются от вашей сущности. От final-класса наследоваться невозможно.
6. **Отсутствие финальных полей/методов:** Поля и методы класса не могут быть final, если они участвуют в маппинге (связывании с колонками БД).
 7. **Наличие первичного ключа (ID):** Класс должен обязательно содержать поле первичного ключа, помеченное аннотацией @Id. Это поле (или группа полей в случае составного ключа) уникально определяет запись в базе данных.
 8. **Инкапсуляция полей:** Поля сущности должны быть напрямую доступны только методам самого класса и скрыты от внешнего мира. Внешний код должен взаимодействовать с ними исключительно через методы доступа (getter / setter методы) или методы бизнес-логики.
 9. **Поддержка сериализации (опционально, но рекомендуется):** Если объект сущности будет передаваться «по значению» за пределы контекста (например, через удаленные интерфейсы, сохраняться в сессии или передаваться по RMI), класс должен реализовывать интерфейс Serializable.

Пример базовой структуры Entity-класса

```
import jakarta.persistence.*;
import java.io.Serializable;

@Entity                                // 1. Указываем, что класс
является сущностью JPA
@Table(name = "students")             // Настройка связи с
конкретной таблицей
public class Student implements Serializable { // Реализация Serializable
при необходимости

    @Id                                // 7. Обязательный
    первичный ключ
    @GeneratedValue(strategy = GenerationType.IDENTITY) // Стратегия
    автоинкремента в БД
    private Long id;
```

```

@Column(name = "first_name", nullable = false, length = 50) // Настройки
к о л о н к и
private String firstName;

@Column(unique = true, nullable = false)
private String email;

// 2. Обязательный конструктор без аргументов
(protected или public)
protected Student() {
}

// Конструктор со всеми параметрами для
удобства разработчика
public Student(String firstName, String email) {
    this.firstName = firstName;
    this.email = email;
}

// 8. Доступ к полям строго через геттеры и
с е т т е р ы
public Long getId() { return id; }
public void setId(Long id) { this.id = id; }

public String getFirstName() { return firstName; }
public void setFirstName(String firstName) { this.firstName = firstName; }

public String getEmail() { return email; }
public void setEmail(String email) { this.email = email; }
}

```

19. Основные аннотации JPA для настройки сущностей: @Table, @Column, @Id, @GeneratedValue. Стратегии генерации первичного ключа.

Для настройки объектно-реляционного отображения (маппинга) Java-классов на таблицы баз данных в JPA используется стандартный набор аннотаций из пакета jakarta.persistence (ранее javax.persistence).

Основные аннотации JPA для настройки сущностей

1. Аннотация *@Table*

- **Назначение:** Указывает на конкретную таблицу в реляционной базе данных, которая сопоставляется с данным Entity-классом.
- **Опциональность:** Является необязательной. Если её не указать, имя таблицы в БД по умолчанию будет в точности совпадать с именем Java-класса.
- **Ключевые параметры:**
 - **name:** Явно задает имя таблицы в базе данных (например, `@Table(name = "users")`).

2. Аннотация *@Column*

- **Назначение:** Используется для детальной настройки свойств столбца таблицы, который связан с конкретным полем Java-класса. Если над полем нет этой аннотации, оно всё равно будет автоматически смappировано по умолчанию.
- **Ключевые параметры:**
 - **name:** Задает имя столбца в таблице, если оно отличается от имени поля в коде (например, `@Column(name = "full_name")`).
 - **nullable:** Определяет, может ли столбец содержать значения null. При `nullable = false` в базе данных генерируется ограничение NOT NULL.
 - **length:** Определяет максимальную длину для строковых типов данных (VARCHAR). По умолчанию составляет 255 символов (например, `length = 100`).
 - **unique:** Указывает, должно ли значение в столбце быть уникальным (`unique = true`), что автоматически создает ограничение уникальности и индекс в структуре БД.

3. Аннотация *@Id*

- **Назначение:** Является обязательной для любой JPA-сущности. Она указывает, что данное поле Java-класса представляет собой **первичный ключ** (PRIMARY KEY) соответствующей таблицы и уникально

идентифицирует каждую строку сущности в контексте сохранения.

4. Аннотация **@GeneratedValue**

- **Назначение:** Используется строго совместно с аннотацией **@Id** и указывает на то, что значение первичного ключа должно генерироваться автоматически на стороне инфраструктуры, а не заполняться разработчиком вручную.
- **Ключевые параметры:**
 - **strategy:** Определяет конкретный подход (стратегию) к генерации идентификаторов с помощью перечисления **GenerationType**.

Стратегии генерации первичного ключа (**GenerationType**)

В спецификации JPA предусмотрено 4 стандартных стратегии генерации ID:

1. **GenerationType.IDENTITY (Автоинкремент)**
 - **Суть:** Генерация уникального ID полностью делегируется встроенному механизму самой базы данных (автоинкрементным колонкам).
 - **Где применяется:** Идеально подходит для СУБД **MySQL** (**AUTO_INCREMENT**) и **PostgreSQL** (**SERIAL / IDENTITY**).
 - **Особенность:** Поскольку генерация происходит непосредственно в момент физической записи строки, **Hibernate** узнает присвоенный ID объекта только *после* фактического выполнения SQL-запроса **INSERT** в базу данных.
2. **GenerationType.SEQUENCE (Последовательность)**
 - **Суть:** Для генерации ключей используется специальный объект базы данных – последовательность (**SEQUENCE**).
 - **Где применяется:** Подходит для СУБД, поддерживающих полноценные последовательности, например, **PostgreSQL** или **Oracle**.

- **Особенность:** Hibernate может заранее запрашивать (резервировать) пачку идентификаторов из последовательности еще до отправки данных. Это делает стратегию наиболее эффективной для пакетных операций вставки (batch insert).
3. GenerationType.TABLE (**Таблица-генератор**)
- **Суть:** В БД создается отдельная служебная таблица, в которой хранится одна строка с текущим максимальным значением ID для всех сущностей приложения. При создании нового объекта фреймворк сначала обновляет число в этой таблице, а затем использует его.
 - **Где применяется:** Используется крайне редко.
 - **Особенность:** Работает универсально на абсолютно любых СУБД, но обладает низкой производительностью из-за постоянных блокировок служебной таблицы при частых запросах.
4. GenerationType.AUTO (**Автоматический выбор**)
- **Суть:** Это стратегия по умолчанию. Hibernate сам анализирует диалект подключенной базы данных и автоматически выбирает наиболее оптимальную для неё стратегию (чаще всего SEQUENCE или IDENTITY).

Пример комплексного использования аннотаций

```
import jakarta.persistence.*;

@Entity // Указывает, что данный класс является JPA-сущностью [cite: 23]
@Table(name = "users") // Переопределяет имя таблицы в БД [cite: 23]
public class User {

    @Id // Поле является первичным ключом таблицы [cite: 23]
    @GeneratedValue(strategy = GenerationType.IDENTITY) // Использование стратегии автоинкремента [cite: 23]
```

```

private Long id;

@Column(name = "full_name", nullable = false, length = 100) // К а с т о м н о е
имя колонки, NOT NULL, длина 100 [cite: 24]
private String name;

@Column(unique = true, nullable = false) // У н и к а л ь н о е з н а ч е н и е ,
з а п р е т н а n u l l
private String email;

@Column(nullable = false) // О г р а н и ч е н и е NOT NULL [cite: 236]
private Integer age;

// К о н с т р у к т о р ы , г е т т е р ы и с е т т е р ы о б я з а т е л ь н ы
д л я р а б о т ы JPA [cite: 19, 236]
protected User() {}
}

```

20. Назначение интерфейса EntityManager. Понятие Persistence Context как кэша первого уровня, его роль в отслеживании изменений.

1. Назначение интерфейса EntityManager

EntityManager (Менеджер сущностей) – это центральный интерфейс спецификации JPA, который инкапсулирует в себе всю логику взаимодействия Java-приложения с контекстом персистентности (и, как следствие, с базой данных). Если JDBC взаимодействует с БД через Connection и Statement, то в JPA вся работа строится вокруг EntityManager.

Основные функции EntityManager:

- **Управление жизненным циклом сущностей:** Перевод объектов между состояниями (Transient, Managed, Detached, Removed) с помощью методов:
 - persist(entity) – сохраняет новый объект в базу данных (переводит в состояние *Managed*).
 - find(Class, id) – ищет объект в БД по первичному ключу.

- `merge(entity)` – обновляет отсоединенный (*Detached*) объект, возвращая его в контекст.
- `remove(entity)` – удаляет запись из базы данных.
- **Создание запросов:** Позволяет писать динамические или статические запросы на языках JPQL (Java Persistence Query Language), Native SQL или с помощью Criteria API (`createQuery()`, `createNativeQuery()`).
- **Управление транзакциями:** Предоставляет доступ к управлению границами транзакций (через метод `getTransaction()`, если не используется Spring-транзакционность).

2. Понятие Persistence Context как кэша первого уровня

Persistence Context (Контекст персистентности) – это своего рода кратковременная «песочница» или внутренняя память, в которой находятся все объекты-сущности, извлеченные из базы данных или сохраненные в ней в рамках текущей транзакции.

В Hibernate контекст персистентности реализуется через объект `Session` и выполняет роль кэша первого уровня (**First-Level Cache**).

Особенности кэша первого уровня:

1. **Время жизни:** Он жестко привязан к текущей транзакции или сессии (вызову `EntityManager`). Как только транзакция завершается или `EntityManager` закрывается, кэш полностью очищается.
2. **Изоляция:** Этот кэш является персональным для каждого потока/запроса. Один пользователь (или поток) никак не может увидеть данные из кэша первого уровня другого пользователя.
3. **Экономия ресурсов:** Если в рамках одной транзакции вы 5 раз вызовете метод `entityManager.find(User.class, 1L)`, реальный SQL-запрос

SELECT в базу данных выполнится только в первый раз. Остальные 4 раза EntityManager мгновенно вернет тот же самый объект из оперативной памяти кэша.

4. **Уникальность объектов (Identity Map):**

Гарантирует, что для одной строки базы данных в памяти будет существовать строго один Java-объект. Сравнение через == двух объектов, загруженных по одному ID в одной транзакции, всегда вернет true.

3. Роль Persistence Context в отслеживании изменений (Dirty Checking)

Один из главных плюсов использования ORM – разработчику не нужно вручную вызывать методы UPDATE или писать SQL для обновления измененных данных. За это полностью отвечает Persistence Context с помощью механизма автоматического отслеживания изменений (в Hibernate это называется **Dirty Checking**).

Как устроен этот процесс под капотом:

1. **Снятие слепка (Snapshot):** Когда EntityManager загружает сущность из базы данных (или когда объект переходит в состояние *Managed*), он сохраняет в своей внутренней памяти две вещи: сам объект и его точную «копию-слепок» (исходное состояние данных из БД).
2. **Модификация в коде:** Разработчик работает с объектом как с обычной Java-моделью: изменяет поля через сеттеры (например, `user.setEmail("new_email@example.com")`). Никаких явных команд вроде `save()` или `update()` вызывать не требуется.
3. **Сравнение (Flush):** В момент завершения транзакции (вызов `commit()`) или при явном вызове `entityManager.flush()`, контейнер сканирует все управляемые объекты в Persistence Context и сравнивает их текущее состояние с сохраненными при загрузке «слепок».
4. **Синхронизация с БД:** Если обнаруживается, что какое-то поле было

изменено (объект стал «грязным» – *dirty*),
Hibernate автоматически генерирует
оптимальный SQL-запрос UPDATE и отправляет
его в базу данных.

Пример кода:

```
@Transactional
public void updateClientAge(Long clientId) {
    // 1. Из БД летит SELECT. Объект попадает в Persistence
    Context, делается слепок.
    Client client = entityManager.find(Client.class, clientId);

    // 2. Меняем поле. В БД ничего сразу не
    отправляется.
    client.setAge(25);

    // Конец метода: Спринг завершает транзакцию.
    // На стадии commit срабатывает Dirty Checking: context видит
    разницу со слепком.
    // Автоматически генерируется и выполняется:
    UPDATE clients SET age = 25 WHERE id = ...
}
```

21. Состояния сущности: Transient (New), Managed (Persistent), Detached, Removed. Объяснить переходы между состояниями при вызове методов persist(), merge(), remove(), detach().

В JPA (Java Persistence API) жизненный цикл сущности описывается четырьмя строго определенными состояниями. Понимание этих состояний и переходов между ними – одна из ключевых тем при работе с ORM (Hibernate), так как от этого зависит, полетят ли запросы в базу данных и изменятся ли в ней строки.

Четыре состояния сущности (Entity States)

1. Transient (или New / Временный)

- **Что значит:** Обычный Java-объект, только что созданный в памяти через оператор `new`.
- **Связь с контейнером:** `EntityManager` ничего не знает об этом объекте, он отсутствует в `Persistence Context` (кэше первого уровня).
- **Связь с БД:** В базе данных соответствующей строки **нет**. Поле `@Id` (если оно автогенерируемое) обычно равно `null`.

2. **Managed (или Persistent / Управляемый)**

- **Что значит:** Объект находится внутри «песочницы» – `Persistence Context`.
- **Связь с контейнером:** `EntityManager` полностью контролирует этот объект. Для него работает механизм **Dirty Checking** (автоматическое отслеживание изменений). Любой вызов сеттера у такого объекта перед коммитом транзакции приведет к автоматическому обновлению строки в БД.
- **Связь с БД:** Объект гарантированно привязан к конкретной строке в таблице (имеет заполненный ID). При этом физическая запись в БД уже существует либо появится сразу после синхронизации (вызова `flush()` или коммита транзакции).

3. **Detached (Отсоединенный)**

- **Что значит:** Объект когда-то был в состоянии `Managed`, у него есть валидный ID и копия данных в БД, но сейчас связь с контекстом разорвана.
- **Связь с контейнером:** `EntityManager` больше не следит за объектом. Если вызывать у него сеттеры, никаких изменений в базе данных автоматически происходить не будет.
- **Связь с БД:** Соответствующая строка в БД существует.

4. **Removed (Удаленный)**

- **Что значит:** Объект находится в процессе удаления.

- **Связь с контейнером:** Он все еще находится в Persistence Context, но помечен специальным флагом «на удаление».
- **Связь с БД:** В момент завершения транзакции (или при flush()) для него выполнится SQL-команда DELETE, и строка сотрется из базы данных.

Переходы между состояниями при вызове методов

Переходы осуществляются с помощью методов интерфейса EntityManager.

1. Метод ***persist(entity)***

- **Переход:** Transient $\rightarrow$ Managed
- **Как работает:** Принимает новый объект. EntityManager берет его под свой контроль (кладет в кэш первого уровня). Если используется стратегия генерации ID IDENTITY, то прямо в этот момент выполняется SQL INSERT, чтобы объект получил идентификатор от СУБД. Если SEQUENCE, то Hibernate просто резервирует ID, а INSERT откладывается до коммита.
- **Важно:** Если передать в persist() объект, который уже находится в состоянии *Detached* (с заполненным ID), выбросится исключение.

2. Метод ***detach(entity)*** (а также ***clear()*** или закрытие контекста)

- **Переход:** Managed $\rightarrow$ Detached
- **Как работает:** Вытаскивает конкретный объект из Persistence Context. EntityManager перестает его отслеживать.
- **Аналоги:** Метод entityManager.clear() переводит абсолютно все управляемые бины текущего контекста в состояние *Detached*. Аналогично происходит при завершении транзакции и закрытии сессии.

3. Метод ***merge(entity)***

- **Переход:** Detached $\rightarrow$ Managed (для копии объекта)

- **Как работает (важнейший нюанс для экзамена):** 1. EntityManager берет отсоединенный объект и смотрит на его @Id.

2. Он ищет объект с таким ID в текущем Persistence Context. Если не находит в памяти – делает запрос SELECT в базу данных и загружает свежую управляемую сущность.

3. Копирует все измененные поля из вашего *Detached*-объекта в этот найденный/загруженный *Managed*-объект.

4. Возвращает ссылку на этот *Managed*-объект.
- **Ловушка:** Исходный объект, который вы передали в аргумент merge(), так и остается в состоянии **Detached!** Управляемой становится только возвращенная копия.

Java

```
// Ошибка: user1 остается detached, изменения поля name
// потеряются
entityManager.merge(user1);
user1.setName("New Name");
```

```
// Правильно: работаем с возвращенной
// управляемой копией
User managedUser = entityManager.merge(user1);
managedUser.setName("New Name"); // Автоматически
```

4. Метод *remove(entity)*

- **Переход:** Managed $\rightarrow$ Removed
- **Как работает:** Помечает сущность на удаление. Физический SQL DELETE сработает только при коммите транзакции. После коммита объект превращается обратно в *Transient* (строки в БД больше нет, но Java-объект еще живет в памяти, пока на него есть ссылки).
- **Важно:** Метод remove() можно вызывать только для управляемых объектов (**Managed**). Если попытаться передать туда *Detached*-объект,

возникнет исключение `IllegalArgumentException`.
Чтобы удалить *Detached*-объект, сначала
нужно сделать его управляемым:
`entityManager.remove(entityManager.merge(detachedEntity));`

Резюме: Как еще можно попасть в состояние **Managed**?

Помимо метода `persist()`, сущности переходят в состояние **Managed** автоматически при любой выборке из базы данных с помощью методов:

- `entityManager.find(Class, id)`
- Выполнение запросов JPQL / Criteria API / Native SQL (`createQuery().getResultList()`).

22. Назначение модуля Spring Data JPA. Объяснить принцип автоматической генерации реализации репозитория на основе интерфейса, наследуемого от `JpaRepository`.

Назначение модуля Spring Data JPA

Spring Data JPA – это дополнительный модуль фреймворка Spring, который представляет собой еще один уровень абстракции *над* спецификацией JPA (Hibernate). Его главная цель – полностью избавить разработчика от написания однотипного шаблонного кода (Boilerplate code) для слоя доступа к данным (DAO / Repository).

Если при классическом использовании JPA вам приходилось вручную внедрять `EntityManager`, открывать транзакции и писать методы вроде `save()`, `findById()` или `update()` для каждой сущности, то Spring Data JPA сводит эту задачу к созданию одного простого интерфейса.

Принцип автоматической генерации реализации репозитория

Когда вы создаете интерфейс и наследуете его от `JpaRepository<User, Long>`, вы не пишете ни одной строки его реализации. Тем не менее, во время работы приложения вы можете успешно вызывать методы `save()`, `findAll()` или кастомные методы вроде `findByEmail(String email)`.

Под капотом Spring проделывает следующую магию:

1. Сканирование пакетов (Component Scanning)

При запуске приложения Spring ищет все интерфейсы, которые наследуются от маркера `Repository` (включая `CrudRepository`, `PagingAndSortingRepository` и `JpaRepository`). За это отвечает аннотация `@EnableJpaRepositories` (в Spring Boot она включается автоматически).

2. Создание динамического прокси (JDK Dynamic Proxy)

Интерфейс сам по себе работать не может – ему нужен класс-реализация. Поскольку на этапе компиляции этого класса не существует, Spring использует встроенный механизм Java – **JDK Dynamic Proxy**.

- В момент поднятия контекста (Runtime) Spring генерирует в оперативной памяти специальный скрытый класс-заглушку (прокси-класс), который реализует ваш интерфейс.
- Этот сгенерированный прокси-класс регистрируется в IoC-контейнере как стандартный Spring-бин. Когда вы пишете `@Autowired private UserRepository repository;`, Spring внедряет именно этот прокси-объект.

3. Делегирование базовых методов (SimpleJpaRepository)

Когда вы вызываете стандартный CRUD-метод (например, `repository.findAll()`), прокси-класс перехватывает этот вызов и перенаправляет его в стандартную базовую реализацию Spring – класс `SimpleJpaRepository`.

Внутри SimpleJpaRepository написан весь тот классический JDBC/JPA код с использованием EntityManager, который вам теперь не нужно писать вручную:

Java

```
// Примерно так выглядит стандартный метод
внутри Spring Data под капотом:
public List<T> findAll() {
    return entityManager.createQuery("select e from " + entityInformation.getEntityName()
+ " e", domainClass).getResultList();
}
```

4. Парсинг кастомных методов (Query Derivation)

Если вы объявляете в интерфейсе собственный метод, например List<User> findByIdAgeGreaterThan(int age);, Spring Data JPA задействует специальный синтаксический анализатор (Query Builder / класс PartTree):

- Он парсит имя метода по ключевым словам (find, By, Age, GreaterThan).
- На основе имени метода Spring автоматически генерирует строковый запрос JPQL: select u from User u where u.age > :age.
- Прокси-класс перехватывает вызов этого метода, подставляет аргумент в сгенерированный JPQL, выполняет его через EntityManager и возвращает вам готовый результат.

Важно для экзамена: Если вы допустите опечатку в названии поля внутри имени метода (например, напишете findByIdAaaage), Spring Data JPA не сможет распарсить структуру класса, и приложение упадет с ошибкой еще на этапе старта контекста, предотвращая runtime-ошибки.

23. Механизм генерации JPQL-запросов на основе имени метода. Привести примеры ключевых слов.

Механизм генерации JPQL-запросов на основе имени метода (Query Derivation) – это встроенная возможность Spring Data JPA, которая позволяет автоматически создавать SQL/JPQL-запросы к базе данных, просто объявляя сигнатуру метода в интерфейсе репозитория по строго определенным правилам наименования.

Разработчику не нужно писать аннотации @Query или использовать EntityManager, если логика выборки укладывается в стандартные правила парсинга.

Как устроен механизм парсинга под капотом

Когда Spring Boot запускает приложение, специальный парсер (класс PartTree) сканирует методы интерфейса репозитория и разбирает их имена на логические составляющие. Имя метода условно делится на две основные части: **Субъект (Subject)** и **Предикат (Predicate)**, разделенные ключевым словом By.

1. **Субъект (до слова By):** определяет, что именно делает метод и какой тип данных он должен вернуть.
 - find..., read..., get..., query... – поиск и возвращение сущностей (одной или списка).
 - count... – подсчет количества строк (возвращает long).
 - exists... – проверка наличия записи (возвращает boolean).
 - delete..., remove... – удаление записей из БД.
 - Также здесь можно указывать модификаторы ограничения, например findFirst3By... или findTopBy....
2. **Слово-разделитель:** By.
3. **Предикат (после слова By):** определяет критерии фильтрации (блок WHERE в SQL). Состоит из имен свойств сущности (полей Java-класса) и специальных ключевых слов, которые связываются логическими операторами And или Or.

Примеры ключевых слов и их маппинг в JPQL

При парсинге свойств Spring Data JPA ищет точное соответствие имени поля в сущности (в стиле *camelCase*).

Ключевое слово	Пример метода в репозитории	Сгенерированный фрагмент JPQL (WHERE ...)
(Равенство)	<code>findByEmail(String email)</code>	<code>where u.email = :email</code>
And	<code>findByNameAndAge(String n, int a)</code>	<code>where u.name = :n and u.age = :a</code>
Or	<code>findByAgeOrStatus(int a, String s)</code>	<code>where u.age = :a or u.status = :s</code>
LessThan / LessThanEqual	<code>findByAgeLessThan(int age)</code>	<code>where u.age < :age</code>
GreaterThan	<code>findBySalaryGreaterThan(double s)</code>	<code>where u.salary > :s</code>
Between	<code>findByAgeBetween(int min, int max)</code>	<code>where u.age between :min and :max</code>
IsNull / IsNotNull	<code>findByMiddleNameIsNull()</code>	<code>where u.middleName is null</code>
Containing / Like	<code>findByNameContaining(String part)</code>	<code>where u.name like %:part%</code>
StartingWith	<code>findByNameStartingWith(String prefix)</code>	<code>where u.name like :prefix%</code>
In	<code>findByStatusIn(Collection<String> s)</code>	<code>where u.status in (:s)</code>

Ключевое слово	Пример метода в репозитории	Сгенерированный фрагмент JPQL (WHERE ...)
True / False	findByActiveTrue()	where u.active = true
IgnoreCase	findByEmailIgnoreCase(String email)	where lower(u.email) = lower(:email)

Сортировка данных (*OrderBy*)

Для сортировки результатов в конец имени метода дописывается ключевое слово `OrderBy`, имя поля и направление сортировки (`Asc` или `Desc`):

- `findByStatusOrderByAgeDesc(String status)`
- **JPQL аналог:** `select u from User u where u.status = :status order by u.age desc`

Важные особенности и ограничения механизма

Критически важно для экзамена: Валидация имен методов происходит строго **в момент старта Spring-контекста**. Если вы допустите опечатку в имени поля (например, в сущности поле называется `firstName`, а в репозитории вы написали `findByFirstname`), приложение **упадет с ошибкой инициализации** на этапе запуска, не дожидаясь вызова метода.

- **Вложенные свойства (Связи):** Если у сущности `User` есть связанная сущность `Address`, у которой есть поле `city`, то метод для поиска пользователей по городу будет выглядеть как `findByAddressCity(String city)` или, для избежания двусмысленности, со знаком подчеркивания: `findByAddress_City(String city)`.
- **Перегрузка аргументов:** Количество параметров в методе должно **строго соответствовать** количеству полей и ключевых слов, указанных в предикате после `By`, и идти в том же порядке.

- Исключение – параметры пагинации (Pageable) или сортировки (Sort), которые всегда можно дописать последним аргументом.
- **Читаемость кода:** Не рекомендуется использовать этот механизм для слишком сложных запросов с большим количеством условий (например, более 3–4 параметров с And/Or). Метод вроде `findByFirstNameAndLastNameAndAgeGreaterThanOrStatusAndEmailContaining` превращается в нечитаемую строку. В таких случаях лучше использовать явную аннотацию `@Query`.

24. Транзакции. Раскрыть каждое свойство ACID с примерами.

Транзакция – это логическая единица работы с базой данных, состоящая из одного или нескольких SQL-запросов, которая воспринимается СУБД как единое целое.

Чтобы гарантировать надежность, предсказуемость и целостность данных при одновременной работе тысяч пользователей, транзакции обязаны соответствовать набору из четырех классических правил – **ACID**.

1. A – Atomicity (Атомарность)

Суть: Принцип «всё или ничего». Транзакция не может быть выполнена частично. Если хотя бы один из SQL-запросов внутри транзакции завершается ошибкой или сбоем, **абсолютно все** предыдущие изменения этой транзакции отменяются (происходит Rollback), и база данных возвращается в исходное состояние.

- **Пример:** Вы переводите 1 000 рублей другу через мобильный банк. Транзакция состоит из двух шагов:
 1. Списать 1 000 рублей с вашего счета.
 2. Зачислить 1 000 рублей на счет друга.

3. *Сценарий сбоя*: Шаг 1 выполнен успешно, но в момент выполнения Шага 2 на сервере банка отключилось электричество. Благодаря атомарности, СУБД отменит Шаг 1. Деньги не испарятся в никуда, а вернутся на ваш счет.

2. C – Consistency (С о г л а с о в а н н о с т ь)

Суть: Транзакция переводит базу данных из одного фиксированного и корректного состояния в другое такое же корректное состояние. Это значит, что транзакция не может нарушить установленные в БД правила, ограничения (Constraints), внешние ключи или бизнес-правила. Если действия транзакции нарушают логику БД, она будет отклонена.

- **Пример:** В структуре БД банка настроено жесткое ограничение: *«Баланс счета не может быть отрицательным»*.
 - *Сценарий сбоя*: У вас на счету 500 рублей, и вы пытаетесь перевести кому-то 1 000 рублей. Попытка выполнить такую транзакцию нарушит правило согласованности (баланс ушел бы в минус 500). СУБД сразу заблокирует эту операцию, сохраняя систему в согласованном и валидном состоянии.

3. I – Isolation (И з о л и р о в а н н о с т ь)

Суть: Параллельно выполняющиеся транзакции не должны влиять друг на друга. Результат выполнения нескольких одновременных транзакций должен быть точно таким же, как если бы они выполнялись строго по очереди, одна за другой. Ни одна транзакция не должна видеть промежуточные (еще не сохраненные) изменения других транзакций.

- **Пример:** Вы и ваша мама одновременно пользуетесь одной семейной картой, на

которой лежит 5 000 рублей. Вы в магазине покупаете куртку за 4 000 рублей, а мама в тот же миг в другом магазине покупает продукты на 2 000 рублей.

- *Сценарий без изоляции:* Обе кассы одновременно считают баланс (5 000), обе одобряют покупку, и баланс уходит в минус.
- *Сценарий с изоляцией:* СУБД изолирует транзакции. Ваша транзакция блокирует строку с балансом, проверяет его, списывает 4 000 рублей и завершается. Только после этого транзакция мамы получает доступ к обновленному балансу (1 000 рублей) и видит, что средств на покупку продуктов за 2 000 уже недостаточно.

4. D – Durability (Долговечность / Стойкость)

Суть: Если транзакция была успешно завершена (пользователь получил подтверждение Commit), её результаты гарантированно сохраняются в энергонезависимой памяти (на жестком диске или SSD). Никакие последующие аварии, внезапные отключения питания или сбои операционной системы не могут уничтожить или откатить эти данные назад.

- **Пример:** Вы успешно оплатили госпошлину на сайте Госуслуг, система написала: *«Оплата прошла успешно»*. Ровно через секунду после этого в дата-центре полностью гаснет свет и отключаются сервера.
 - Когда сервера перезагрузятся, СУБД восстановит свое состояние по специальному журналу упреждающей записи (WAL – Write-Ahead Log). Ваша оплата не потеряется и останется зафиксированной в системе.

25. Назначение аннотации @Transactional. Описать поведение Spring при успешном завершении метода и при возникновении исключения. Настройки rollbackFor и noRollbackFor

Назначение аннотации @Transactional

Аннотация @Transactional используется для декларативного управления транзакциями в Spring. Вместо того чтобы вручную писать громоздкий код открытия, фиксации или отката транзакции (через EntityManager или Connection), разработчик просто помечает аннотацией метод или целый класс.

Под капотом Spring использует механизмы AOP (аспектно-ориентированного программирования): он оборачивает ваш бин в специальный класс-прокси, который перехватывает вызовы метода и автоматически управляет транзакционным контекстом.

Поведение Spring при завершении метода

Когда управление внутри метода, помеченного @Transactional, завершается, прокси-класс анализирует исход этого выполнения:

1. При успешном завершении метода

Если метод отработал до конца без исключений (или выполнил оператор return), прокси-класс перехватывает управление и:

- Вызывает метод flush() у контекста персистентности, чтобы синхронизировать все изменения

объектов в памяти с базой данных (генерация SQL-запросов).

- Вызывает команду `commit()` у менеджера транзакций (`PlatformTransactionManager`), окончательно сохраняя данные в БД.
- Закрывает соединение и возвращает его в пул соединений.

2. При возникновении исключения (Поведение по умолчанию)

Если внутри метода (или в вызываемых им методах) произошло исключение, прокси ловит его, и здесь кроется главный нюанс для экзамена:

- **Unchecked Exceptions (Не проверяемые):** Если выброшено исключение, наследуемое от `RuntimeException` (например, `NullPointerException`, `IllegalArgumentException`, `DataAccessException`) или `Error`, Spring автоматически вызывает `rollback()` (откат). Все изменения, сделанные в рамках этой транзакции, аннулируются.
- **Checked Exceptions (Проверяемые):** Если выброшено исключение, наследуемое напрямую от `Exception` (например, `IOException`, `SQLException`), Spring по умолчанию **НЕ отменяет транзакцию, а делает `commit()` для всех успешных шагов, выполненных до этой ошибки.**

Настройки `rollbackFor` и `noRollbackFor`

Чтобы изменить стандартное поведение Spring и гибко управлять логикой отката при специфических ошибках, у аннотации `@Transactional` есть два важных параметра:

1. *`rollbackFor`*

- **Зачем нужен:** Позволяет явно указать классы исключений, при возникновении которых транзакцию **необходимо откатить** (даже если по умолчанию Spring бы этого не сделал, например, для проверяемых исключений).
- **Пример применения:** Если в вашем методе может вылететь проверяемое исключение

FileNotFoundException или CustomCheckedException, но бизнес-логика требует отменить все операции с базой при его возникновении.

Java

```
// Транзакция откатится, если выбросится любое
// проверяемое исключение Exception
@Transactional(rollbackFor = Exception.class)
public void processOrder() throws Exception {
    // логика работы с БД
    if (errorOccurred) {
        throw new Exception("Проверяемая ошибка, но
        транзакция всё равно откатится");
    }
}
```

2. *noRollbackFor*

- **Зачем нужен:** Позволяет указать классы исключений, при возникновении которых транзакцию **НЕ нужно откатывать** (даже если это RuntimeException, который по умолчанию приводит к откату). Транзакция будет успешно зафиксирована (commit).
- **Пример применения:** Вы регистрируете попытку входа пользователя. Метод должен записать лог в базу данных, но если пароль неверный, выбрасывается кастомное BadCredentialsException (наследник RuntimeException). Вам важно, чтобы запись о неудачной попытке *сохранилась* в БД, а не откатилась.

Java

```
// Транзакция закоммитится, несмотря на то, что
// выброшен RuntimeException
@Transactional(noRollbackFor = InsufficientFundsException.class)
public void withdrawMoney() {
    // Логика: списываем комиссию за проверку
    // баланса (запись в БД должна остаться!)
    billingService.chargeFee();

    // Выбрасывается RuntimeException, но благодаря noRollbackFor
    // комиссия не откатится
    throw new InsufficientFundsException("Недостаточно средств для
    основной операции");
}
```

26. Параметры readOnly и propagation аннотации @Transactional.

Объяснить влияние `readOnly = true` на производительность. Сравнить значения `Propagation.REQUIRED` и `Propagation.REQUIRES_NEW`.

Параметр `readOnly` аннотации `@Transactional`

Параметр `readOnly = true` сообщает Spring и лежащему под ним провайдеру ORM (Hibernate), что данный метод будет выполнять только операции чтения (SELECT) и не будет изменять данные в базе.

Влияние `readOnly = true` на производительность:

1. **Отключение Dirty Checking (самый главный плюс):** Когда сущности загружаются в Persistence Context в режиме Read-Only, Hibernate не делает слепки (snapshots) этих объектов в памяти. В момент завершения транзакции фреймворку не нужно сканировать кэш первого уровня и сравнивать объекты с их первоначальным состоянием. Режим очистки выставляется в `FlushMode.MANUAL`, что экономит огромное количество оперативной памяти и ресурсов CPU при обработке больших массивов данных.
2. **Оптимизации на уровне СУБД:** Spring передает этот флаг прямо в JDBC-драйвер. Многие СУБД (например, Oracle или PostgreSQL) оптимизируют выполнение таких транзакций: они могут выделять под них изолированные пулы, отключать генерацию логов отката (Undo Logs) или полностью отказываться от наложения блокировок на таблицы/строки, что ускоряет параллельный доступ.
3. **Маршрутизация запросов (Read-Replicas):** В крупных проектах с разделением баз данных на Master (для записи) и Replica (для чтения) флаг `readOnly = true` часто

используется инфраструктурой для автоматического перенаправления тяжелых поисковых запросов на реплики, разгружая основную базу данных.

Параметр propagation (Распространение транзакций)

Параметр propagation определяет границы транзакции и поведение Spring в ситуации, когда один транзакционный метод вызывает другой транзакционный метод.

Сравнение Propagation.REQUIRED и Propagation.REQUIRES_NEW

Это две самые популярные и часто используемые стратегии распространения.

К р и т е р и й	Propagation.REQUIRED (По умолчанию)	Propagation.REQUIRES_NEW
Главный принцип	«Присоединит ься к существующей »	«Всегда создавать новую»
Поведение , если внешней транзакции и НЕТ	Spring создает новую транзакцию.	Spring создает новую транзакцию.
Поведение , если внешняя транзакция ЕСТЬ	Внутренний метод НЕ создает свою транзакцию, а выполняется в рамках уже существующей внешней.	Внешняя транзакция приостанавливае тся (suspend). Создается вторая, абсолютно новая и независимая транзакция.
Влияние отката	Если во внутреннем	Если внутренний метод падает,

К р и т е р и й	Propagation. REQUIRED (По умолчанию)	Propagation. REQUIRES_NEW
внутренне го метода	методе происходит ошибка, вся общая транзакция помечается как rollback-only. Даже если внешний метод перехватит ошибку в try- catch, сохранить внешние данные не удастся – упадет всё.	откатывается только его транзакция. Если внешний метод корректно обрабатывает это исключение через try-catch, то внешняя транзакция сможет успешно завершиться (commit).
Влияние отката внешнего метода	Если внутренний метод отработал успешно, но в конце внешнего метода произошел сбой, откатятся абсолютно все изменения (и внешние, и внутренние), так как транзакция была одна на двоих.	Если внутренний метод завершился успешно, его транзакция сразу же фиксируется (commit) в БД. Последующий сбой во внешнем методе никак не затронет уже сохраненные внутренним методом данные.

*Пример классического применения **REQUIRES_NEW**:*

Логирование операций (аудит). Вам необходимо записать в базу данных информацию о том, что пользователь попытался совершить покупку. Если сама покупка сорвется (произойдет откат бизнес-логики), запись в таблице логов аудита **всё равно должна остаться**. Поэтому метод записи логов помечают как **REQUIRES_NEW**.

27. Проблема N+1 запроса. Механизм её возникновения. Как решить?

Проблема N+1 запроса: Суть и механизм возникновения

Проблема N+1 запроса – это одна из самых частых проблем производительности при использовании ORM-фреймворков (Hibernate / JPA). Она заключается в том, что вместо одного эффективного запроса к базе данных для извлечения связанных данных приложение выполняет **один** начальный запрос для получения списка главных сущностей, а затем **N дополнительных запросов** для загрузки связанных данных для каждой из N строк.

Как это происходит на практике (Механизм):

Представим две сущности: Post (Пост) и его Comment (Комментарии). Связь – Один ко многим (@OneToMany). По умолчанию для коллекций используется ленивая загрузка (FetchType.LAZY).
Java

@Entity

```
public class Post {
```

```
    @Id @GeneratedValue private Long id;
```

```
    private String title;
```

```
    @OneToMany(mappedBy = "post", fetch = FetchType.LAZY)
```

```
    private List<Comment> comments;
```

```
}
```

Разработчик хочет вывести список всех постов и количество комментариев к каждому из них:

Java

```
// Вызываем метод репозитория, выполняющий
обычный findAll()
List<Post> posts = postRepository.findAll();

for (Post post : posts) {
    // В этот момент Hibernate видит обращение к LAZY-
коллекции
    // и вынужден делать отдельный SQL-запрос для
каждого поста!
    System.out.println(post.getTitle() + " - " + post.getComments().size());
}
```

Что происходит на уровне базы данных:

1. 1-й запрос (запрос «1»): Выборка всех постов.

SQL

```
SELECT * FROM post; — Возвращает, например, N = 100
постов
```

2. Последующие запросы (запрос «N»):
Итерация по циклу заставляет Hibernate для каждого конкретного post_id делать точечный запрос комментариев:

SQL

```
SELECT * FROM comment WHERE post_id = 1;
SELECT * FROM comment WHERE post_id = 2;
— ... и так еще 98 раз!
```

Итог: Вместо 1 запроса приложение сделало 101 запрос. Если постов в базе будет 10 000, приложение просто заблокирует базу данных огромным количеством мелких запросов.

Важно для экзамена: Ошибка часто возникает и при FetchType.EAGER (для связей @ManyToOne). Если вы используете обычный JPQL-запрос select p from Post p, Hibernate сначала получит посты, а затем, обнаружив тип EAGER, тут же начнет в

цикле догружать связанные сущности теми же самыми N запросами.

Способы решения проблемы N+1

1. Использование *JOIN FETCH* в JPQL (Самый частый способ)

Мы явно приказываем Hibernate выполнить SQL-оператор LEFT JOIN или INNER JOIN и сразу же («жадно») заполнить коллекцию связанных объектов в рамках одного-единственного запроса.

- В репозитории Spring Data:

```
public interface PostRepository extends JpaRepository<Post, Long> {  
    @Query("SELECT p FROM Post p LEFT JOIN FETCH p.comments")  
    List<Post> findAllWithComments();  
}
```

- SQL под капотом:

```
SELECT p.*, c.* FROM post p LEFT JOIN comment c ON c.post_id = p.id;
```

Результат: Ровно 1 запрос к базе данных.

2. Использование аннотации *@EntityGraph*

Это декларативный способ от Spring Data JPA, позволяющий динамически менять стратегию загрузки полей с ленивой на «жадную» для конкретного метода, не переписывая JPQL-запросы вручную.

Java

```
public interface PostRepository extends JpaRepository<Post, Long> {  
    @EntityGraph(attributePaths = {"comments"})  
    List<Post> findAll(); // Spring сам построит JOIN FETCH для  
    свойства comments  
}
```

3. Настройка *@Fetch(FetchMode.SUBSELECT)* (Специфика Hibernate)

Эта аннотация ставится над ленивой коллекцией в самом Entity-классе.

Java

```
@OneToMany(mappedBy = "post")  
@Fetch(FetchMode.SUBSELECT)  
private List<Comment> comments;
```

- **Как работает:** При первом обращении к комментариям любого из постов Hibernate выполнит **всего один** дополнительный запрос, который выкачает комментарии сразу для *всех* ранее загруженных постов с помощью подзапроса.
- **Итог:** Вместо $1 + N$ запросов мы получаем строго **2 запроса** (один для постов, один для всех их комментариев).

4. Использование ограничения `@BatchSize`

Позволяет указать Hibernate, какими «порциями» (батчами) следует подгружать ленивые сущности или коллекции.

Java

```
@OneToMany(mappedBy = "post")
@BatchSize(size = 20)
private List<Comment> comments;
```

- **Как работает:** Вместо генерации запроса для одного поста, Hibernate соберет ID первых 20 постов и выполнит запрос с оператором IN: `SELECT * FROM comment WHERE post_id IN (1, 2, ... 20);`.
- **Итог:** Количество запросов сокращается с $1 + N$ до $1 + \lceil N / 20 \rceil$. Этот метод хорош как глобальная страховка производительности приложения.

28. Типы связей между сущностями. Привести примеры.

В JPA (Java Persistence API) связи между сущностями отражают реальные отношения между таблицами в реляционных базах данных. Всего выделяют **4 основных типа связей**, каждый из которых настраивается соответствующей аннотацией.

Связи могут быть **однонаправленными** (Unidirectional – когда одна сущность знает о другой, а обратно ссылки нет) и **двунаправленными** (Bidirectional – когда обе сущности ссылаются друг на друга).

1. Один к одному (@OneToOne)

- **Суть:** Одной записи в первой таблице соответствует ровно одна запись во второй таблице.
- **Реализация в БД:** В одной из таблиц создается колонка с внешним ключом (FOREIGN KEY), ссылающаяся на первичный ключ другой таблицы. Для обеспечения связи «один к одному» на эту колонку обязательно накладывается ограничение уникальности (UNIQUE).
- **Пример: Пользователь (User) и его Паспорт (Passport).** У одного пользователя может быть только один паспорт, и конкретный паспорт принадлежит строго одному человеку.

Java

@Entity

```
public class User {
```

```
    @Id @GeneratedValue private Long id;
```

```
    private String name;
```

```
    @OneToOne(cascade = CascadeType.ALL)
```

```
    @JoinColumn(name = "passport_id", referencedColumnName = "id")
```

```
    private Passport passport; // Ссылка на связанную сущность
```

```
}
```

2. Многие к одному (@ManyToOne)

- **Суть:** Множество записей из таблицы А могут ссылаться на одну и ту же запись в таблице Б. Это самый популярный и эффективный тип связи в базах данных.
- **Реализация в БД:** На стороне таблицы «Многие» (таблица А) создается колонка с внешним ключом (FOREIGN KEY), которая указывает на первичный ключ таблицы «Один» (таблица Б).
- **Пример: Сотрудник (Employee) и Департамент (Department).** Много разных сотрудников могут работать в одном и том же отделе.

Java

@Entity

```
public class Employee {
```

```
    @Id @GeneratedValue private Long id;
```

```
    private String name;
```

```

    @ManyToOne
    @JoinColumn(name = "department_id") // Внешний ключ будет в
таблице сотрудников
    private Department department;
}

```

3. Один ко многим (@OneToMany)

- **Суть:** Обратная сторона связи «Многие к одному». Одной записи в таблице А соответствует целая коллекция записей из таблицы Б.
- **Важный нюанс для экзамена:** В реальных проектах одиночную (однонаправленную) связь @OneToMany используют редко. Если написать её без параметров, Hibernate создаст лишнюю промежуточную таблицу связей, что снижает производительность. Поэтому её почти всегда объединяют со связью @ManyToOne на другой стороне, делая связь **двунаправленной**.
- Чтобы указать, что связь зеркальная, используется параметр mappedBy (он указывает на имя поля в противоположном классе).
- **Пример:** Пост в блоге (Post) и Комментарии (Comment). У одного поста может быть список (коллекция) из множества комментариев.

Java

@Entity

public class Post {

@Id @GeneratedValue private Long id;

private String title;

// mappedBy показывает, что поле "post" в классе Comment является главным в этой связи

@OneToMany(mappedBy = "post", cascade = CascadeType.ALL)

private List<Comment> comments = new ArrayList<>();

}

4. Многие ко многим (@ManyToMany)

- **Суть:** Множество записей из таблицы А могут быть связаны со множеством записей из таблицы Б.
- **Реализация в БД:** Реляционные базы данных не умеют связывать таблицы

таким образом напрямую. Поэтому под капотом ORM-фреймворк автоматически создает третью, служебную связующую таблицу (Join Table). Она состоит всего из двух колонок – внешних ключей, связывающих ID первой и второй таблиц.

- **Пример: Студент (Student) и Курс (Course).** Один студент может записаться на множество разных курсов, в то же время на одном курсе учится множество студентов.

Java

@Entity

```
public class Student {
```

```
    @Id @GeneratedValue private Long id;
```

```
    private String name;
```

```
    @ManyToMany
```

```
    @JoinTable(
```

```
        name = "student_courses", // Имя служебной таблицы в БД
```

```
        joinColumns = @JoinColumn(name = "student_id"), // Ключ этой
```

```
сущности
```

```
        inverseJoinColumns = @JoinColumn(name = "course_id") // Ключ
```

```
связанной сущности
```

```
)
```

```
    private List<Course> courses = new ArrayList<>();
```

```
}
```

Резюме для ответа на экзамене:

1. @OneToOne – уникальный внешний ключ в одной из таблиц.
2. @ManyToOne – стандартный внешний ключ на стороне «многих». **Владелец связи.**
3. @OneToMany – коллекция на стороне «одного». Использует mappedBy, чтобы не плодить лишних таблиц.
4. @ManyToMany – всегда порождает под капотом третью связующую таблицу (Join Table).

29. Назначение параметров cascade, orphanRemoval, mappedBy в аннотациях связей. Понятие «владелец связи».

При работе с двунаправленными (Bidirectional) связями в JPA возникает ситуация, когда две

сущности ссылаются друг на друга в Java-коде, но в реляционной базе данных связь может физически управляться только с одной стороны. Для настройки этого поведения и автоматизации управления жизненным циклом связанных объектов используются параметры `mappedBy`, `cascade` и `orphanRemoval`.

1. Понятие «Владелец связи» (Relationship Owner)

В реляционной БД связь между таблицами всегда хранится в одном конкретном месте – там, где находится колонка с внешним ключом (FOREIGN KEY). Однако в Java-коде при двунаправленной связи ссылки есть у обоих классов.

Чтобы Hibernate не запутался, какую именно ссылку считать истинной при генерации SQL, вводится понятие **Владельца связи**:

- **Владелец связи (Owning side)** – это сущность, которая «отвечает» за внешние ключи в базе данных. Именно её состояние анализирует Hibernate при сохранении или обновлении данных.
 - *Как определить:* В коде владелец связи **всегда** определяется отсутствием параметра `mappedBy` (обычно над его полем стоит аннотация `@JoinColumn`). В связях «Один ко многим» владельцем всегда является сторона «Многие» (`@ManyToOne`).
- **Обратная сторона (Inverse side)** – это пассивная сторона связи. Изменения в её полях полностью игнорируются фреймворком при синхронизации с БД. Она нужна только для удобства чтения данных в коде (чтобы у родителя можно было вызвать метод `.getChildren()`).

Важнейшее правило для экзамена: Если вы добавите объект в коллекцию на *обратной стороне* связи, но не установите ссылку на *стороне владельца*, Hibernate **не запишет** связь в базу данных, и внешний ключ останется `null`.

2. Назначение параметра mappedBy

Параметр mappedBy используется для указания того, что текущее поле является **обратной (пассивной) стороной** связи.

- **Значение параметра:** В качестве значения передается **строковое имя поля** в противоположном классе-владельце, которое осуществляет этот маппинг.
- **Где применяется:** Используется в аннотациях @OneToMany, @ManyToMany и пассивной стороне @ManyToOne. В аннотации @ManyToOne этого параметра **не существует**, так как эта сторона априори является владельцем.

Пример:

Java

@Entity

```
public class Post { // ОБРАТНАЯ СТОРОНА (Не владеет связью)
```

```
    @Id @GeneratedValue private Long id;
```

```
    // mappedBy указывает на поле "post" в классе Comment
```

```
    @OneToMany(mappedBy = "post")
```

```
    private List<Comment> comments;
```

```
}
```

@Entity

```
public class Comment { // ВЛАДЕЛЕЦ СВЯЗИ (Управляет внешним ключом)
```

```
    @Id @GeneratedValue private Long id;
```

```
    @ManyToOne
```

```
    @JoinColumn(name = "post_id") // Физическая колонка FK в
```

```
таблице comment
```

```
    private Post post;
```

```
}
```

3. Назначение параметра cascade (Каскадирование)

Каскадирование (Cascade) определяет, должны ли операции над родительской сущностью (сохранение, удаление, обновление) автоматически распространяться (каскадироваться) на все связанные с ней дочерние сущности.

Параметр принимает массив значений из перечисления `CascadeType`:

- `CascadeType.PERSIST` – при сохранении родителя (`entityManager.persist()`) автоматически сохраняются все новые дочерние объекты из коллекции.
- `CascadeType.MERGE` – при обновлении родителя автоматически обновляются (мержаются) дочерние объекты.
- `CascadeType.REMOVE` – при удалении родителя из БД автоматически удаляются все связанные с ним дочерние записи.
- `CascadeType.REFRESH` / `CascadeType.DETACH` – транслируют операции обновления состояния и отсоединения от контекста на связанные сущности.
- `CascadeType.ALL` – объединяет в себе абсолютно все вышеперечисленные типы каскадирования.

4. Назначение параметра `orphanRemoval` (Удаление «сирот»)

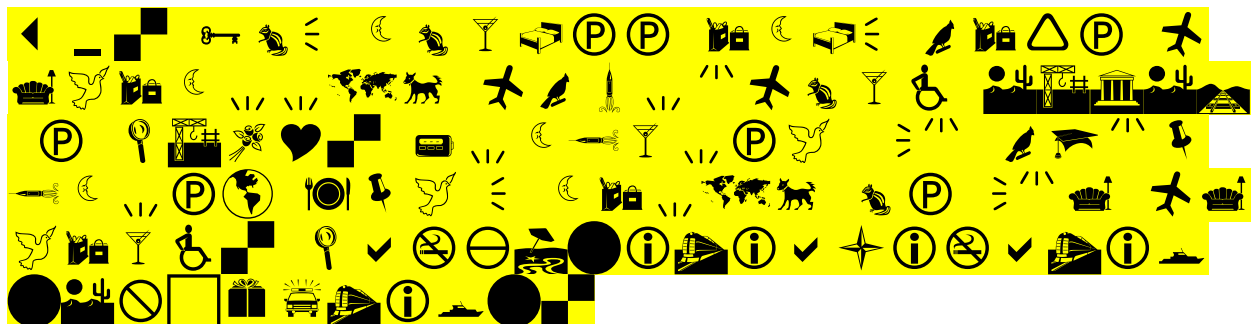
Параметр `orphanRemoval = true` (доступен для `@OneToMany` и `@OneToOne`) включает автоматическое удаление дочерних записей из базы данных, если они **теряют связь** со своим родителем (становятся «сиротами»).

- **Как это работает:** Если вы просто удалите дочерний объект из коллекции родителя (например, `post.getComments().remove(0);`), Hibernate поймет, что этот комментарий больше не связан с постом, автоматически сгенерирует SQL-запрос `DELETE` и сотрет эту строку из таблицы комментариев.

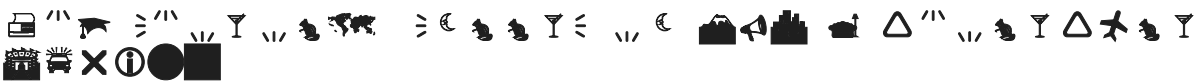
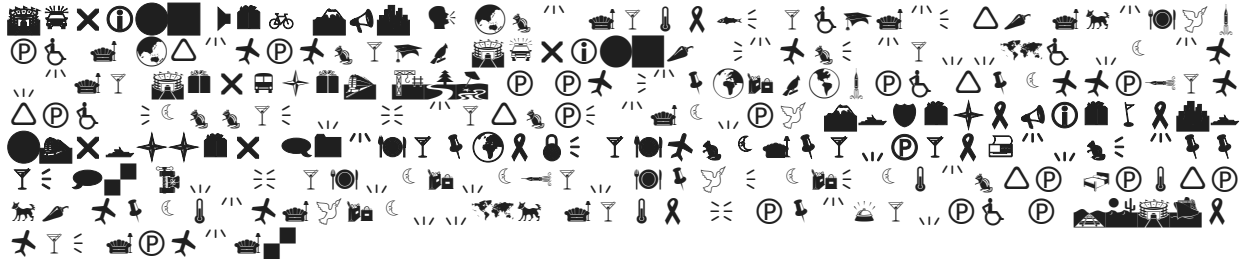
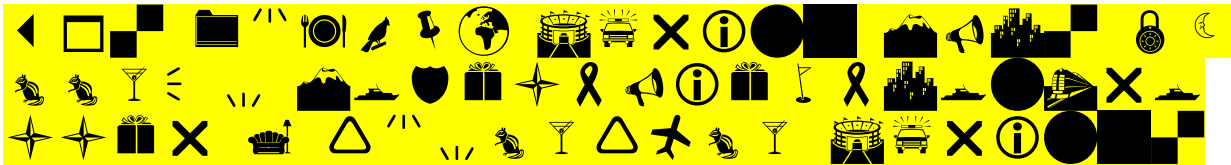
*Главное отличие: **`CascadeType.REMOVE`** против **`orphanRemoval = true`***

Очень часто на собеседованиях и экзаменах просят сравнить эти две настройки. Их поведение различается в корне:

Ситуация	CascadeType. REMOVE	orphanRemoval = true
Удаляем целиком объект Post	Удалит все связанные комментарии из БД.	Удалит все связанные комментарии из БД.
Удаляем один Comment из коллекции внутри Post	Связь разорвется. В БД у комментария post_id станет равным null (если разрешено), но сама строка останется жить.	Комментарий превращается в «сироту». Строка физически удаляется из базы данных через DELETE.



- Key, bird, moon, sun, clouds, trees, animals (dog, cat, bird), and various geometric shapes and symbols (circles, squares, triangles, arrows, infinity symbols).
- Key, bird, moon, sun, clouds, trees, animals (dog, cat, bird), and various geometric shapes and symbols (circles, squares, triangles, arrows, infinity symbols).
- Key, bird, moon, sun, clouds, trees, animals (dog, cat, bird), and various geometric shapes and symbols (circles, squares, triangles, arrows, infinity symbols).



- A large, dense block of black icons on a white background, featuring a wide variety of symbols related to technology, environment, and social issues.
- A large, dense block of black icons on a white background, featuring a wide variety of symbols related to technology, environment, and social issues.
- A large, dense block of black icons on a white background, featuring a wide variety of symbols related to technology, environment, and social issues.



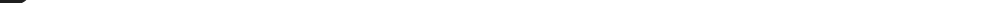
[illegible]

This image displays a comprehensive collection of black icons arranged in a grid-like fashion. The icons are diverse, covering categories such as nature (e.g., sun, moon, clouds, trees), technology (e.g., smartphone, laptop, Wi-Fi symbol), social media (e.g., heart, speech bubble, share icon), and everyday objects (e.g., car, house, shopping bag). The icons are rendered in a simple, minimalist style, making them easily recognizable and suitable for various digital applications.

[illegible][illegible]

[illegible]

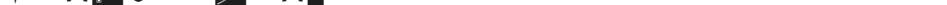
- [illegible]

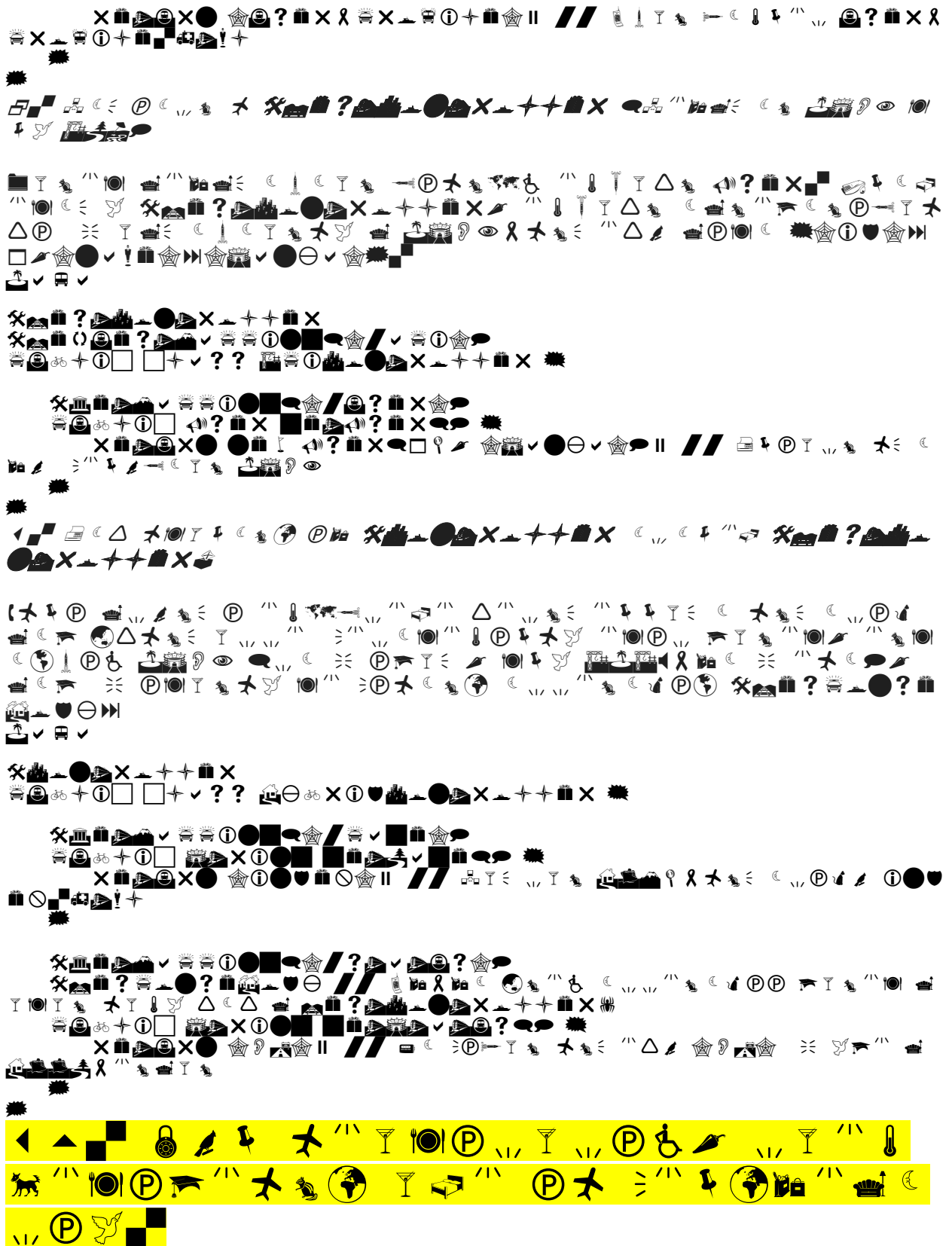


[illegible]

-
- A large grid of 100 small icons representing various symbols, including weather, animals, objects, and abstract shapes.

A large grid of 100 icons representing various concepts, arranged in 10 rows and 10 columns. The icons include symbols for nature, animals, objects, and abstract shapes.

- 



Что такое пул соединений (Connection Pool)

Пул соединений (Connection Pool) – это специальный буфер (кэш) в оперативной памяти приложения, в котором заранее созданы и поддерживаются в активном состоянии

готовые соединения (подключения) с базой данных.

Вместо того чтобы создавать новое физическое подключение при каждом запросе к БД и закрывать его сразу после завершения операции, приложение берет уже существующее соединение из пула, использует его и возвращает обратно в пул для повторного использования другими потоками.

В мире Java и Spring Boot стандартом де-факто является пул соединений **HikariCP** – он подключается автоматически «из коробки» и известен своей экстремальной скоростью работы.

Проблема: Жизнь без пула соединений

Если бы пула соединений не существовало, жизненный цикл каждого запроса к базе данных выглядел бы так:

1. **Установка сетевого соединения:** Открытие TCP-сокета между сервером приложения и сервером СУБД (требует времени на сетевой «рукопожатие» – TCP handshake).
2. **Аутентификация и авторизация:** База данных должна проверить логин и пароль пользователя, а также его права доступа на выполнение операций.
3. **Выделение ресурсов на стороне СУБД:** Под каждый коннект база данных создает отдельный процесс или поток и выделяет ему кусок оперативной памяти.
4. **Выполнение SQL-запроса.**
5. **Закрытие соединения:** Освобождение ресурсов и закрытие сетевого сокета.

Главный минус: Шаги 1, 2, 3 и 5 в сумме занимают в разы больше времени, чем само выполнение SQL-запроса (Шаг 4). При высокой нагрузке (например, 1000 запросов в секунду) база данных просто откажет в обслуживании, так как захлебнется в

процессе постоянного создания и уничтожения тяжелых системных потоков.

Решение: Как работает пул соединений

Пул соединений полностью меняет эту схему. При старте Spring Boot приложения инициализируется бин DataSource, управляющий пулом (например, HikariCP), и сразу создает фиксированное количество подключений (например, 10 штук по умолчанию).

Когда вашему репозиторию нужно выполнить запрос, происходит следующее:

[Поток приложения] —(1. Просит коннект)—> [ПУЛ СОЕДИНЕНИЙ]
[Поток приложения] <—(2. Получает коннект)— [Активные соединения]
[(готовы к работе)]
[Поток приложения] —(3. Выполняет SQL)—> [БАЗА ДАННЫХ (БД)]
[Поток приложения] —(4. Возвращает назад)—> [ПУЛ СОЕДИНЕНИЙ]

1. Метод (например, с аннотацией @Transactional) запрашивает соединение у пула.
2. Пул моментально выдает **уже открытое** и авторизованное соединение. Поток не тратит время на авторизацию и сеть.
3. Приложение выполняет SQL-запрос.
4. По завершении метода вместо физического закрытия соединения (connection.close()) вызывается логическое закрытие – объект просто освобождается и **возвращается обратно в пул**.

Необходимость использования пула (Главные преимущества)

- **Максимальная производительность (Низкая задержка / Latency):** Время на получение коннекта из пула составляет доли миллисекунд, тогда как создание нового физического соединения может занимать от десятков до сотен миллисекунд.
- **Экономия ресурсов СУБД:** База данных не перегружается постоянным созданием и

удалением системных процессов. Она работает со стабильным, предсказуемым количеством открытых сессий.

- **Защита от перегрузок (Throttling / Ограничение нагрузки):** Пул имеет максимальный лимит соединений (параметр `maximum-pool-size`). Если в систему одновременно придет больше пользователей, чем этот лимит, новые запросы не обрушат базу данных, а просто встанут в безопасную очередь ожидания (настроенную через `connection-timeout`).
- **Контроль «мертвых» соединений:** Пул сам следит за качеством соединений. Если сеть моргнула или база данных закрыла коннект по таймауту, пул автоматически удалит испорченное соединение и создаст вместо него новое, не беспокоя код разработчика.

Перед вами краткая, структурированная «шпаргалка» по всем 35 вопросам, оптимизированная для быстрого повторения и сдачи экзамена.

ЧАСТЬ 1: Инструменты сборки (Maven и Gradle)

1. Принцип «Соглашение над конфигурацией» в Maven. Стандартная структура каталогов

- **Суть принципа:** Инструмент предлагает готовые настройки по умолчанию (где лежат файлы, куда собирать артефакты). Разработчику не нужно писать конфигурацию, если он следует этим правилам.
- **Структура каталогов:**
 - `src/main/java` – исходный код приложения.
 - `src/main/resources` – конфигурационные файлы, свойства, ресурсы.
 - `src/test/java` – исходный код тестов.
 - `src/test/resources` – ресурсы для тестов.
 - `target/` – директория, куда Maven складывает результаты сборки (компилированный код, `.jar` файлы).

- pom.xml – главный конфигурационный файл проекта в корне.

2. Координаты Maven (groupId, artifactId, version).

Правила именования

- groupId: Уникальный идентификатор организации или создателя.
 - *Правило:* Записывается в стиле обратного доменного имени (мелкий регистр, через точку). *Пример:* com.example.fitquest.
- artifactId: Имя конкретного проекта или его отдельного модуля.
 - *Правило:* Мелкий регистр, слова разделяются дефисами. *Пример:* fitness-tracker-backend.
- version: Версия проекта.
 - *Правило:* Формат X.Y.Z (Мажорная.Минорная.Патч). Если проект в разработке, добавляется суффикс -SNAPSHOT. *Пример:* 1.0.0-SNAPSHOT.

3. Стандартный жизненный цикл сборки Maven. Основные фазы

Жизненный цикл состоит из последовательных фаз. Вызов любой фазы автоматически выполняет все предыдущие:

1. **validate** – проверяет, корректен ли проект и заполнена ли вся информация.
2. **compile** – компилирует исходный код (.java → .class).
3. **test** – запускает модульные (unit) тесты.
4. **package** – упаковывает скомпилированный код в архив (.jar или .war).
5. **verify** – проводит интеграционные тесты для проверки качества.
6. **install** – копирует собранный пакет в локальный репозиторий (папка .m2), чтобы его могли использовать другие локальные проекты.
7. **deploy** – отправляет готовый артефакт в удаленный репозиторий (например, Nexus) для общего доступа.

4. Механизм управления транзитивными зависимостями в Maven. Области видимости (Scopes)

- **Механизм:** Если ваш проект зависит от библиотеки А, а библиотека А зависит от библиотеки Б, Maven автоматически скачает библиотеку Б (она станет транзитивной). При конфликтах версий используется правило *«ближайшего в дереве зависимостей»* (выигрывает та версия, которая ближе к корню проекта).
- **Области видимости (Scopes):**
 - `compile` (по умолчанию) – зависимость доступна везде (при компиляции, тестах, сборке и работе).
 - `provided` – нужна для компиляции и тестов, но в рантайме её предоставит среда исполнения (например, контейнер сервлетов типа Tomcat). Не включается в итоговый `.jar`.
 - `runtime` – не нужна для компиляции, но необходима при запуске программы (например, драйвер базы данных JDBC).
 - `test` – доступна только при компиляции и запуске тестов (например, JUnit, Mockito).

5. Структура многомодульного проекта Maven. Parent POM vs Child POM. Секции `dependencyManagement` и `dependencies`

- **Структура:** Один корневой (родительский) проект координирует сборку вложенных подпроектов (модулей).
- **Parent POM:** Содержит глобальные настройки, общие плагины и версии библиотек. Находится в корне.
- **Child POM:** Наследует настройки родителя, описывает специфичные зависимости конкретного модуля.
- **Различия секций:**
 - `<dependencyManagement>` – используется **только в Parent POM**. Она просто декларирует

(фиксирует) версии библиотек, но *не скачивает их*.

- `<dependencies>` – используется везде. В дочерних модулях вы объявляете нужную библиотеку без указания версии, и Maven скачивает её, беря версию из родительского `dependencyManagement`.

6. Модель Gradle (Project, Task, Action). Фазы жизненного цикла Gradle

- **Модель:** Gradle представляет сборку как направленный ациклический граф задач.
 - **Project** – собираемый компонент (модуль). У каждого проекта есть файл `build.gradle`.
 - **Task** – атомарная единица работы в проекте (например, компиляция, создание `jar`).
 - **Action** – конкретный блок исполняемого кода (код на Groovy/Kotlin) внутри задачи.
- **Фазы жизненного цикла:**
 1. **Initialization (Инициализация):** Gradle определяет, какие проекты (модули) участвуют в сборке (читает `settings.gradle`).
 2. **Configuration (Конфигурация):** Создаются объекты `Project`, выполняются скрипты сборки, выстраивается граф зависимостей между задачами (`Tasks`). Код вне `Actions` выполняется здесь.
 3. **Execution (Исполнение):** Выполняются только те задачи, которые были запрошены пользователем, в соответствии с графом.

7. Конфигурации зависимостей Gradle

- `implementation` – зависимость доступна текущему модулю, но скрыта от других модулей, которые зависят от текущего (не течет транзитивно, ускоряет сборку).
- `api` (требует плагина `java-library`) – зависимость доступна текущему модулю и транзитивно передается всем модулям-потомкам.

- `compileOnly` – аналогично `provided` в Maven. Нужна только для компиляции.
- `runtimeOnly` – нужна только при запуске программы.
- `testImplementation` – библиотека нужна исключительно для написания и запуска тестов.

ЧАСТЬ 2: Spring Framework (Core и IoC Container)

8. Принцип Inversion of Control (Инверсия управления)

- **Суть принципа:** Программа передает контроль над созданием объектов, управлением их жизненным циклом и связыванием зависимостей внешнему фреймворку (контейнеру), вместо того чтобы делать это вручную.
- **Решаемые проблемы традиционного подхода:**
 - *Жесткая связанность (Tight Coupling):* Использование ключевого слова `new` привязывает код к конкретной реализации класса, а не к интерфейсу.
 - *Сложность тестирования:* Невозможно подменить реальный тяжелый объект (например, сервис отправки почты) на фейковый (Mock-объект) в модульных тестах.
 - *Дублирование кода:* Ручное управление глубокими деревьями зависимостей приводит к громоздкому коду инициализации.

9. Внедрение зависимостей (Dependency Injection) как способ реализации IoC. Типы внедрения

- **Суть DI:** Контейнер сам передает (внедряет) требуемые зависимости в объект при его создании.
- **Типы внедрения:**
 1. **Через конструктор (Constructor Injection):**
Рекомендуемый способ. Гарантирует неизменяемость (`final` поля), объект

нельзя создать в некорректном (неполном) состоянии, код легко тестировать.

2. **Через сеттер (Setter Injection):**

Используется для необязательных или динамически изменяемых зависимостей.

3. **Внедрение в поле (Field Injection):**

Внедрение через `@Autowired` прямо над приватным полем. Считается антипаттерном (сильная привязка к Spring, скрытые зависимости, сложность юнит-тестов без рефлексии).

10. Понятия «Бин», «Контейнер», «ApplicationContext»

- **Бин (Bean):** Обычный Java-объект, который создан, настроен и управляется IoC-контейнером Spring.
- **Контейнер (IoC Container):** Ядро фреймворка, которое отвечает за весь жизненный цикл бинов – от их парсинга из конфигурации до уничтожения.
- **ApplicationContext:** Главный интерфейс Spring-контейнера. Он расширяет базовый фабричный интерфейс (`BeanFactory`) и добавляет интеграцию с AOP, обработку событий, интернализацию и специфику веб-приложений.

11. Основные стадии жизненного цикла бина в контейнере. `@PostConstruct` и `@PreDestroy`

1. **Instantiating** – создание экземпляра Java-класса (вызов конструктора).
2. **Populate Properties** – внедрение зависимостей (DI через поля, сеттеры или конструктор).
3. **Aware Interfaces** – бин узнает свое имя в контейнере, контекст и т.д. (вызовы методов `BeanNameAware`, `ApplicationContextAware`).
4. **BeanPostProcessor (Before Initialization)** – предварительная обработка другими компонентами.
5. **Initialization:**

- Вызов метода, помеченного `@PostConstruct` (используется для кастомной инициализации, когда зависимости уже внедрены).
 - Вызов метода `afterPropertiesSet()` интерфейса `InitializingBean`.
6. **BeanPostProcessor (After Initialization)** – финальная обработка, здесь бин может быть обернут в AOP-прокси. Бин готов к работе.
7. **Destruction:** При закрытии контекста вызывается метод с аннотацией `@PreDestroy` (используется для безопасного закрытия ресурсов, очистки пулов, закрытия сетевых соединений).

12. Скоупы (Области видимости) бинов. Различия Singleton и Prototype

- **singleton** (по умолчанию) – один экземпляр бина на весь IoC-контейнер. Все запросы к контейнеру возвращают один и тот же объект.
- **prototype** – при каждом запросе бина из контейнера (или его внедрении в другой бин) создается абсолютно новый экземпляр.
- **Веб-скоупы:** `request` (один бин на один HTTP-запрос), `session` (один бин на одну HTTP-сессию).
- **Особенность уничтожения prototype-бинов:** Spring создает `prototype`-бин, выполняет его инициализацию и отдает пользователю. После этого контейнер не отслеживает его жизненный цикл. Метод с аннотацией `@PreDestroy` для `prototype`-бинов никогда не вызывается автоматически. Очищать ресурсы такого объекта программист должен вручную.

13. Алгоритм разрешения зависимостей при `@Autowired`. `@Primary` и `@Qualifier`

- **Алгоритм `@Autowired`:**
 1. Spring ищет подходящий бин по типу (by Type).

2. Если найден ровно один бин, он внедряется.
 3. Если найдено несколько бинов одного типа, Spring пытается выбрать бин по его имени (by Name), сравнивая имя бина с именем переменной.
 4. Если имя не помогло разрешить неоднозначность, выбрасывается исключение `NoUniqueBeanDefinitionException`.
- **Способы решения конфликта:**
 1. `@Primary` – ставится над одним из классов-реализаций. Говорит контейнеру: «Если есть выбор, по умолчанию всегда выбирай этот бин».
 2. `@Qualifier("beanName")` – ставится в точке внедрения рядом с `@Autowired`. Явно указывает строковое имя конкретного бина, который нужно внедрить.

14. Способы конфигурирования Spring-контейнера

1. **Конфигурация на Java-классах (Java-based):** Использование классов с аннотацией `@Configuration` и методов с `@Bean`. *Применение:* Подключение сторонних библиотек, где нельзя поставить аннотацию над классом, и сложная условная логика создания бинов.
2. **Аннотации и сканирование компонентов (Annotation-based):** Использование аннотаций `@Component`, `@Service`, `@Repository`, `@Controller` над классами вместе с `@ComponentScan`. *Применение:* Основной способ для написания собственной бизнес-логики приложения.
3. **XML-конфигурация:** Описание бинов в файлах `.xml`. *Применение:* Устаревший подход, встречается только в старых легаси-проектах.

ЧАСТЬ 3: Базы данных, JPA и Hibernate

15. Назначение JDBC. Недостатки прямого использования JDBC

- **Назначение:** Стандартный низкоуровневый Java API для отправки SQL-запросов и получения результатов из любых реляционных баз данных.
- **Недостатки:**
 - *Шаблонный код (Boilerplate):* Огромное количество рутины по открытию/закрытию Connection, Statement, ResultSet и обработке исключений.
 - *Ручной маппинг:* Разработчик обязан вручную перекладывать данные из строк таблицы (ResultSet) в Java-объекты.
 - *Отсутствие переносимости:* SQL-запросы пишутся под конкретный диалект СУБД, менять базу данных тяжело.
 - *Проверяемые исключения:* Необходимость везде обрабатывать низкоуровневый SQLException.

16. Технология Object-Relational Mapping (ORM)

ORM – это концепция, которая сглаживает разницу между объектно-ориентированным кодом Java и реляционной структурой таблиц БД:

- Класс Java преобразуется в **Таблицу** в БД.
- Поле объекта преобразуется в **Колонку** (столбец) таблицы.
- **Экземпляр (объект)** класса преобразуется в конкретную **Строку** таблицы.
- **Связи между объектами** (ссылки) преобразуются во **Внешние ключи (Foreign Keys)** или ассоциативные таблицы.

17. Спецификация JPA vs Реализация Hibernate

- **JPA (Jakarta Persistence API):** Это абстрактная спецификация, стандарт, просто набор интерфейсов и правил (документация), описывающий, как должно работать ORM-решение в Java.
- **Hibernate:** Это конкретная библиотека, полноценный программный продукт, который **реализует** спецификацию JPA.

Hibernate воплощает интерфейсы JPA в рабочий код и предоставляет дополнительные расширенные возможности, которых нет в стандарте.

18. Сущность (Entity). Правила создания JPA-сущности

Entity – это обычный Java-класс, отображаемый на таблицу в базе данных. Правила создания:

1. Класс обязательно должен быть помечен аннотацией `@Entity`.
2. Должен иметь первичный ключ, помеченный аннотацией `@Id`.
3. Обязательно наличие конструктора без параметров (`public` или `protected`) – он нужен Hibernate для воссоздания объектов из БД через рефлексию.
4. Класс и его методы не должны быть `final` (иначе Hibernate не сможет создать прокси-объект для ленивой загрузки).

19. Основные аннотации JPA. Стратегии генерации первичного ключа

- `@Table(name = "...")` – задает имя таблицы в БД (если имя класса отличается).
- `@Column(name = "...", nullable = false)` – настраивает параметры конкретной колонки (имя, уникальность, запрет на `null`).
- `@Id` – объявляет поле первичным ключом.
- `@GeneratedValue(strategy = ...)` – задает стратегию генерации ID:
 - `IDENTITY` – генерация силами самой БД через встроенный автоинкремент (удобно для MySQL, PostgreSQL).
 - `SEQUENCE` – использование специального объекта базы данных – последовательности (`Sequence`). Самый эффективный вариант для Hibernate.
 - `TABLE` – эмуляция генерации через отдельную служебную таблицу (очень медленная стратегия, устарела).
 - `AUTO` – Hibernate сам выбирает стратегию на основе используемой СУБД.

20. Интерфейс EntityManager. Понятие Persistence Context

- **EntityManager**: Главный интерфейс JPA для взаимодействия с базой данных (CRUD-операции: `persist`, `merge`, `remove`, `find`).
- **Persistence Context (Контекст персистентности)**: Это кратковременное внутреннее хранилище («песочница»), где находятся все объекты, загруженные или сохраненные через EntityManager в рамках текущей транзакции.
 - *Роль кэша 1-го уровня*: Если в рамках одной транзакции дважды запросить объект с ID=1, Hibernate не будет делать второй SQL-запрос, а отдаст объект из контекста.
 - *Отслеживание изменений (Dirty Checking)*: Контекст помнит исходное состояние объекта. В конце транзакции он сравнивает текущее состояние объекта с исходным и сам генерирует SQL-запросы UPDATE для измененных полей, без явного вызова методов сохранения со стороны разработчика.

21. Состояния сущности (Entity States) и переходы между ними

Сущность может находиться в одном из 4 состояний:

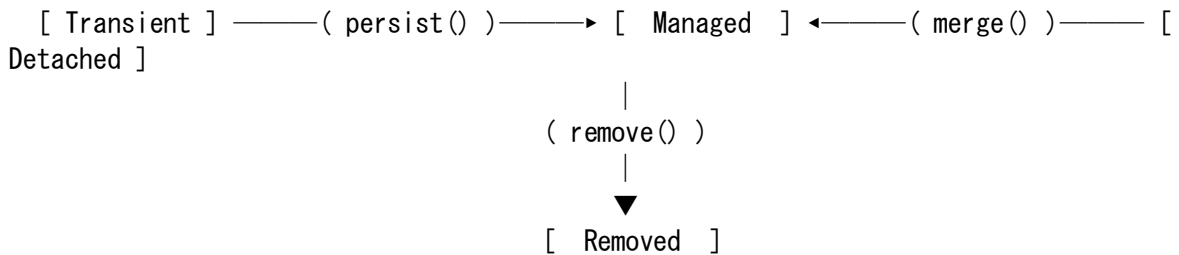
1. **Transient (New)**: Объект только что создан через `new`. Он живет в памяти Java, но контекст персистентности и БД о нем ничего не знают. У него нет ID.
2. **Managed (Persistent)**: Объект привязан к контексту персистентности и имеет ID. Любое изменение его полей будет автоматически засинкено в БД.
 - *Переход*: Из Transient в Managed через метод `persist()`, или из Detached через метод `merge()`.
3. **Detached (Отсоединенный)**: Объект имеет ID в БД, но связь с контекстом персистентности разорвана (например,

транзакция закрылась). Изменения полей больше не отслеживаются.

- *Переход*: Из Managed в Detached через метод detach() или clear().

4. **Removed (Удаленный)**: Объект все еще находится в контексте персистентности, но помечен на удаление. В конце транзакции в БД улетит запрос DELETE.

- *Переход*: Из Managed в Removed через метод remove().



22. Назначение Spring Data JPA. Принцип генерации репозиториев

- **Назначение**: Модуль Spring, который кардинально упрощает слой доступа к данным (DataAccess Layer), полностью избавляя разработчика от ручного написания кода репозиториев и вызовов EntityManager.
- **Принцип генерации**: Разработчик пишет обычный интерфейс и наследует его от встроенного JpaRepository<Entity, IdType>. При старте приложения Spring находит этот интерфейс и с помощью механизмов **Dynamic Proxy** (динамических прокси) на лету генерирует его рабочую реализацию прямо в оперативной памяти, регистрируя её как стандартный бин.

23. Механизм генерации JPQL-запросов на основе имени метода. Ключевые слова

Spring Data JPA парсит (разбирает) имя метода интерфейса по специальным правилам, вычлняя префиксы и условия, и автоматически строит по ним валидный JPQL/SQL запрос.

- *Примеры ключевых слов*:

- findByLastName(String name) $\rightarrow$ WHERE last_name = ?
- findByAgeGreaterThan(int age) $\rightarrow$ WHERE age > ?
- existsByEmail(String email) $\rightarrow$ Проверка существования записи (возвращает boolean).
- countByStatus(String status) $\rightarrow$ SELECT COUNT(*)... WHERE status = ?

ЧАСТЬ 4: Транзакции, Spring MVC и Пул соединений

24. Транзакции. Свойства ACID с примерами

Транзакция – неделимая группа SQL-операций. Свойства:

- **A – Atomicity (Атомарность):** Принцип «всё или ничего». Если один шаг упал, откатывается вся транзакция. *Пример:* Перевод денег. Списали у вас, но не зачислили другу из-за сбоя – деньги вернутся вам (Rollback).
- **C – Consistency (Согласованность):** Транзакция не может нарушить правила и ограничения БД. *Пример:* Баланс счета не может быть отрицательным. Попытка уйти в минус будет заблокирована СУБД.
- **I – Isolation (Изолированность):** Параллельные транзакции не видят промежуточных изменений друг друга. *Пример:* Две одновременные покупки по одной карте с балансом 5 000 рублей пройдут строго по очереди, не дав уйти балансу в минус.
- **D – Durability (Долговечность):** Если транзакция закоммичена, данные сохранены на диск навечно. *Пример:* Свет погас через секунду после оплаты – после перезагрузки сервера данные восстановятся из лога транзакций.

25. Аннотация @Transactional. Поведение Spring при успехе и ошибках. rollbackFor и noRollbackFor

- **Назначение:** Декларативное управление транзакциями через AOP-прокси.

- **Поведение при успешном завершении:** Spring вызывает `commit()`, сохраняя все изменения в базу данных, и закрывает транзакцию.
- **Поведение при исключении:**
 - *Unchecked Exception* (наследники `RuntimeException` и `Error`) – Spring автоматически делает `rollback()` (откат).
 - *Checked Exception* (проверяемые исключения) – Spring по умолчанию **НЕ откатывает** транзакцию, а делает `commit()`.
- **Настройки:**
 - `rollbackFor` – указывает проверяемые исключения, при которых *нужно* принудительно делать откат. *Пример:* `@Transactional(rollbackFor = Exception.class)`.
 - `noRollbackFor` – указывает непроверяемые исключения, при которых откат делать *не нужно*.

26. Параметры `readOnly` и `propagation`. Влияние `readOnly = true` на перформанс. `REQUIRED` vs `REQUIRES_NEW`

- **Параметр `readOnly = true`:** Указывает, что транзакция только читает данные.
 - *Влияние на производительность:* Hibernate отключает механизм `Dirty Checking` (не делает слепки объектов в памяти), очистка кэша сбрасывается в `Manual`. СУБД оптимизирует блокировки строк, что сильно ускоряет работу памяти и CPU.
- **Сравнение `Propagation` (распространение):**
 - `Propagation.REQUIRED` (по умолчанию) – если внешняя транзакция есть, метод присоединяется к ней. Если нет – создает новую. Всё выполняется в одной транзакции.
 - `Propagation.REQUIRES_NEW` – всегда приостанавливает внешнюю транзакцию и создает свою собственную, независимую. Ошибка во внутренней транзакции не сорвет внешнюю, если её перехватить через `try-catch`.

27. Проблема N+1 запроса. Механизм возникновения и решения

- **Механизм:** Возникает, когда приложение делает 1 запрос для получения списка родительских сущностей (например, 100 постов), а затем в цикле делает N дополнительных запросов для ленивой (LAZY) подгрузки дочерних данных (комментариев) для каждого родителя по отдельности. Итог: вместо 1 запроса выполняется 101.
- **Как решить:**
 1. Использовать JOIN FETCH в JPQL-запросе (загрузит всё за 1 запрос через SQL JOIN).
 2. Использовать аннотацию @EntityGraph над методом репозитория.
 3. Поставить над коллекцией аннотацию @Fetch(FetchMode.SUBSELECT) (сократит количество запросов до 2).
 4. Использовать @BatchSize для подгрузки данных пачками (батчами).

28. Типы связей между сущностями с примерами

1. @OneToOne (**Один к одному**): Одной строке соответствует ровно одна строка.
Пример: Пользователь (User) и его Паспорт (Passport).
2. @ManyToOne (**Многие к одному**): Много записей ссылаются на одну. *Пример:* Сотрудники (Employee) работают в одном Департаменте (Department). Является владельцем связи.
3. @OneToMany (**Один ко многим**): Обратная сторона связи. *Пример:* У одного Поста (Post) есть коллекция Комментариев (Comment). Обычно настраивается с параметром mappedBy.
4. @ManyToMany (**Многие ко многим**): Множество объектов связаны со множеством других.
Пример: Студенты (Student) и Курсы (Course).
Всегда порождает промежуточную таблицу связей (Join Table).

29. Параметры cascade, orphanRemoval, mappedBy. Понятие «Владелец связи»

- **Владелец связи:** Сущность, которая управляет внешним ключом (FOREIGN KEY) в базе данных (в коде у неё нет параметра mappedBy). Изменения именно её полей приводят к обновлению связей в БД.
- **mappedBy:** Указывает, что поле является зеркальной (пассивной) стороной связи. Принимает имя поля из класса-владельца.
- **cascade (Каскадирование):** Переносит операции (сохранение, удаление, обновление) с родительского объекта на все связанные дочерние. (Например, CascadeType.ALL).
- **orphanRemoval = true:** Автоматически физически удаляет дочернюю запись из таблицы БД, если её просто удалили из Java-коллекции родительского объекта (превратили в «сироту»).

30. Стратегии загрузки EAGER и LAZY.

Дефолты. LazyInitializationException

- **EAGER (Жадная):** Данные подгружаются сразу с основным объектом через SQL JOIN. *Дефолт для:* @ManyToOne, @OneToOne.
- **LAZY (Ленивая):** Вместо реальных данных подставляется прокси-заглушка. Данные догружаются отдельным запросом только при первом вызове геттера поля. *Дефолт для:* @OneToMany, @ManyToMany.
- **LazyInitializationException:** Ошибка возникает, когда код пытается прочитать ленивое (LAZY) поле объекта, но транзакция (сессия Hibernate) уже закрылась. Решение: загружать данные жадно через JOIN FETCH или @EntityGraph до закрытия транзакции.

31. Модуль Spring MVC. Паттерн Model-View-Controller в контексте Spring

- **Spring Web MVC** – фреймворк для создания веб-приложений на основе сервлетов.

- **Компоненты паттерна:**
 - **Model (Модель):** Данные приложения (Java-объекты, POJO, карты Map), передаваемые для отображения.
 - **View (Представление):** Шаблон интерфейса (Thymeleaf, HTML) или формат данных (JSON/XML), видимый пользователю.
 - **Controller (Контроллер):** Класс с аннотацией @Controller, который принимает HTTP-запрос, вызывает сервисы бизнес-логики и решает, какую модель и какое представление вернуть.

32. DispatcherServlet в Spring MVC, его функции

DispatcherServlet – это центральный сервлет, реализующий паттерн **Front Controller** (единая точка входа). Он выступает главным диспетчером и оркестратором: принимает абсолютно все входящие HTTP-запросы и координирует другие компоненты для формирования ответа.

33. Назначение компонентов HandlerMapping, HandlerAdapter, ViewResolver

- **HandlerMapping** – ищет по URL запроса подходящий контроллер и метод, который должен этот запрос обработать.
- **HandlerAdapter** – берет найденный метод контроллера, подготавливает для него аргументы из HTTP-запроса и физически вызывает этот метод через рефлексию.
- **ViewResolver** – берет строковое логическое имя страницы, вернувшееся из контроллера (например, "index"), и превращает его в реальный путь к файлу шаблона (например, /templates/index.html).

34. Различия между аннотациями @Controller и @RestController

- **@Controller** – используется для классических веб-страниц. Методы возвращают String (имя

HTML-шаблона), который обрабатывается компонентом `ViewResolver`.

- `@RestController` – это гибрид: `@Controller` + `@ResponseBody`. Используется для **REST API**. Методы возвращают чистые Java-объекты/DTO, которые автоматически преобразуются библиотекой *Jackson* в формат **JSON** и пишутся прямо в тело HTTP-ответа.

35. Пул соединений (Connection Pool), необходимость использования

- **Что это:** Выделенный буфер в памяти (например, библиотека **HikariCP**), содержащий заранее созданные, открытые и авторизованные соединения с базой данных.
- **Необходимость использования:** Создание нового физического TCP-соединения с СУБД и прохождение авторизации при каждом запросе пользователя – это крайне долгая и ресурсоемкая операция. Пул соединений позволяет мгновенно выдавать готовые коннекты потокам приложения и забирать их обратно, что в разы увеличивает скорость работы системы и защищает базу данных от перегрузки.